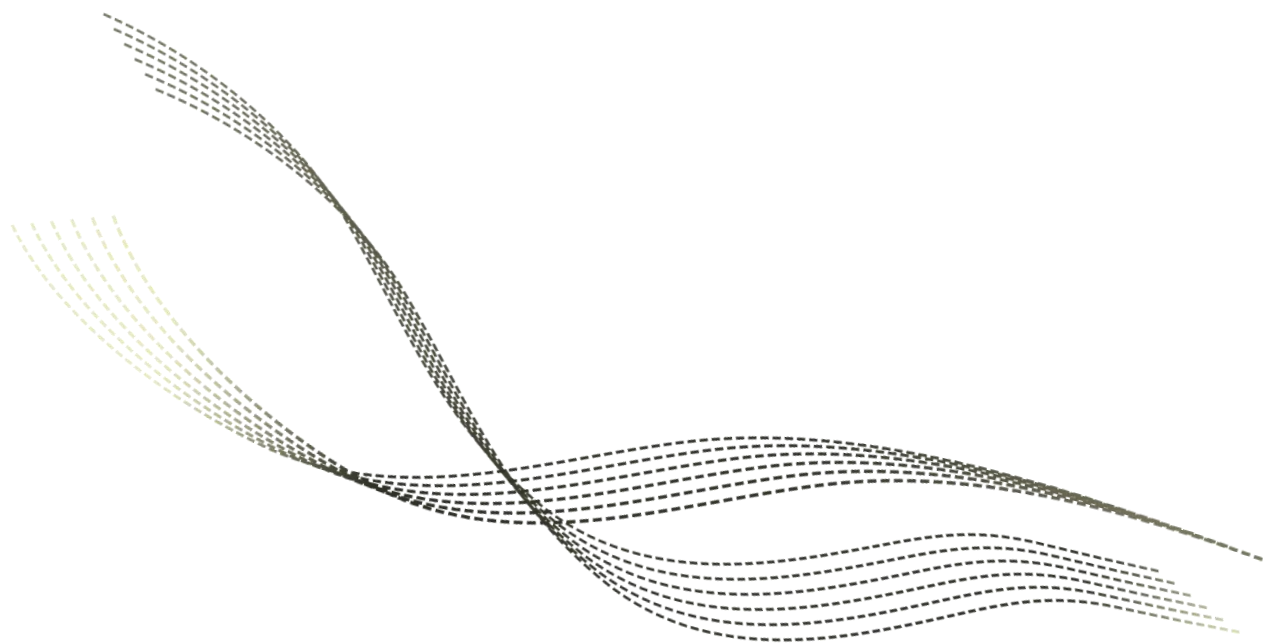


超大规模集成电路设计导论

Very Large Scale Integration Circuit (VLSI) Design

集成电路布局方法

沈明华



目录

CONTENTS

01

布局问题

02

模拟退火

03

图划分

04

解析式布局

PART 04

解析式布局

① 二次函数优化布局

■ 布局算法历史

1970-1980	1980-1990	1990-2010		>2010		
基于划分	模拟退火	最小割 (多层次)	解析式		解析式	
			二次型	非线性	二次型	非线性
Breuer	Timberwolf VPR	FengShui	GORDIAN	APlace	POLAR	ePlace Replace
Dunlop & Kernighan	Dragon	Capo	BonnPlace	Naylor Synopsis	SimPL ComPLx	DREAMPlace
Quadratic Assignment		Capo+Roos ter	mFar	NTUplace	MAPLE	
Cadence QPlace			Kraftwerk	mPL6		
低质量	低效率		FastPlace			
			Warp3			

■ 布局算法历史

1970-1980	1980-1990	1990-2010			>2010	
基于划分	模拟退火	最小割 (多层次)	解析式		解析式	
			二次型	非线性	二次型	非线性
Breuer	Timberwolf VPR	FengShui	GORDIAN	APlace	POLAR	ePlace Replace
Dunlop & Kernighan	Dragon	Capo	BonnPlace	Naylor Synopsis	SimPL ComPLx	DREAMPlace
Quadratic Assignment		Capo+Rooster	mFar	NTUplace	MAPLE	
Cadence QPlace			Kraftwerk	mPL6		
			FastPlace			
			Warp3			

■ 二次型

□ 问题定义

– 输入

- 模块(包括标准器件和宏模块) $B_1, B_2, \dots, B_n$
- 每个模块的形状和引脚(pin)的位置
- 连接模块的导线(nets) $N_1, N_2, \dots, N_m$

– 输出

- 模块的坐标 (x, y)
- 要求在给定的区域内，没有模块发生重叠

– 优化目标

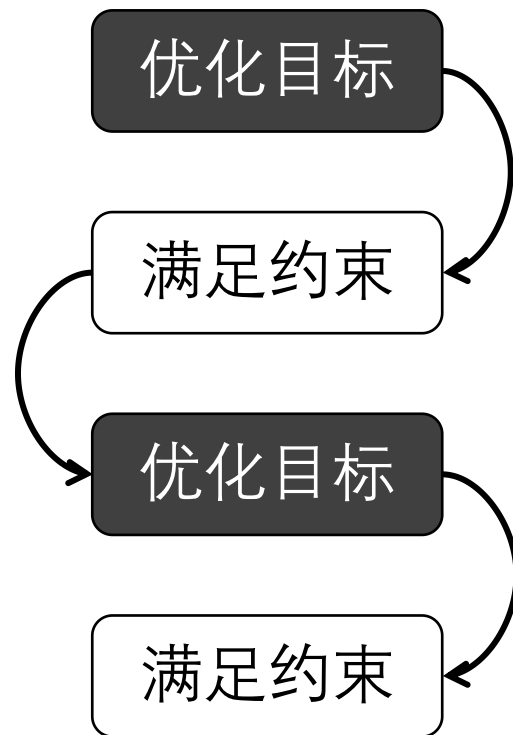
- 总线长
 - 建模为二次方的目标函数，然后使用函数优化技术优化目标函数

■ 二次型

□ 一般二次型函数优化算法的流程

– 采用迭代优化流程

1. 以总线长为目标，调用优化算法来优化布局
2. 消除布局中的重叠，满足布局约束
3. 如果目标函数已经收敛，则退出算法
4. 回到步骤1



■ 二次型

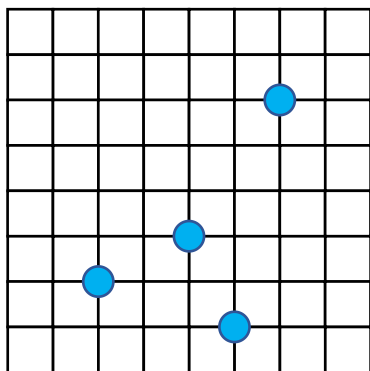
□ 算法总览

- 下面介绍的算法在线长优化方面大差不差，用梯度法解一个二次方程。主要区别在于如何消除模块重叠
- SimPL
 - 采用增加虚拟模块的策略，利用相对分散的虚拟网表，达到消去模块重叠的目的
- Gordian
 - 采用划分策略，逐步限制模块布局范围

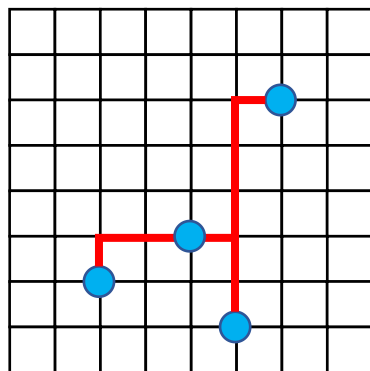
■ 二次型

□ 怎样计算总线长

– 复习：前面提到三种曼哈顿距离模型

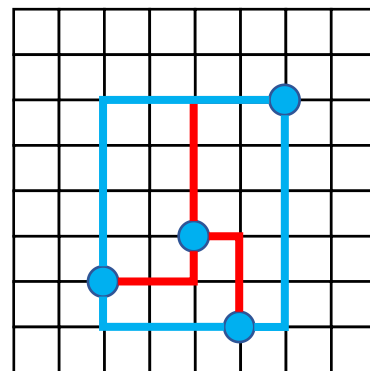


假设一个连接4个引脚的线网

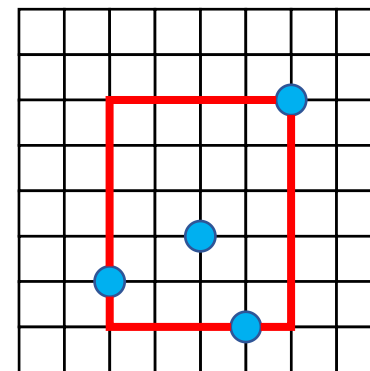


最小steiner树模型

$$\sum |x_i - x_j| + \sum |y_i - y_j| \quad \max |x_i - x_j| + \max |y_i - y_j|$$



团(clique)模型



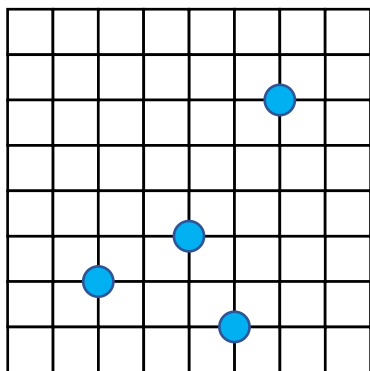
半周长(HPWL)模型

属于曼哈顿距离模型

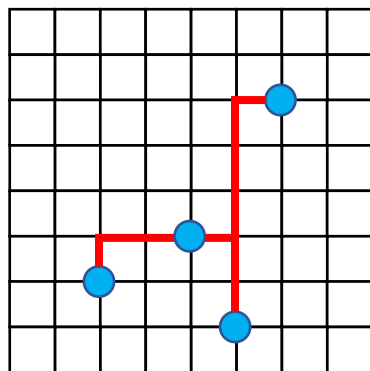
■ 二次型

□ 怎样计算总线长

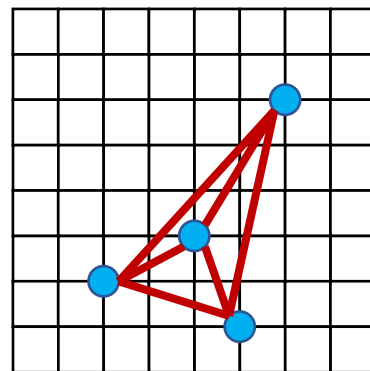
- 除了曼哈顿线长预估模型，还有两种欧式线长预估模型
- 二次型解析式布局算法的目标函数基本上是欧式距离模型



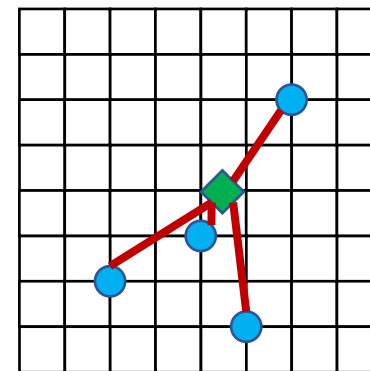
假设一个连接4个引脚的线网



最小steiner树模型



团(clique)模型



星型

$$\sum (x_i - x_j)^2 + \sum (y_i - y_j)^2$$

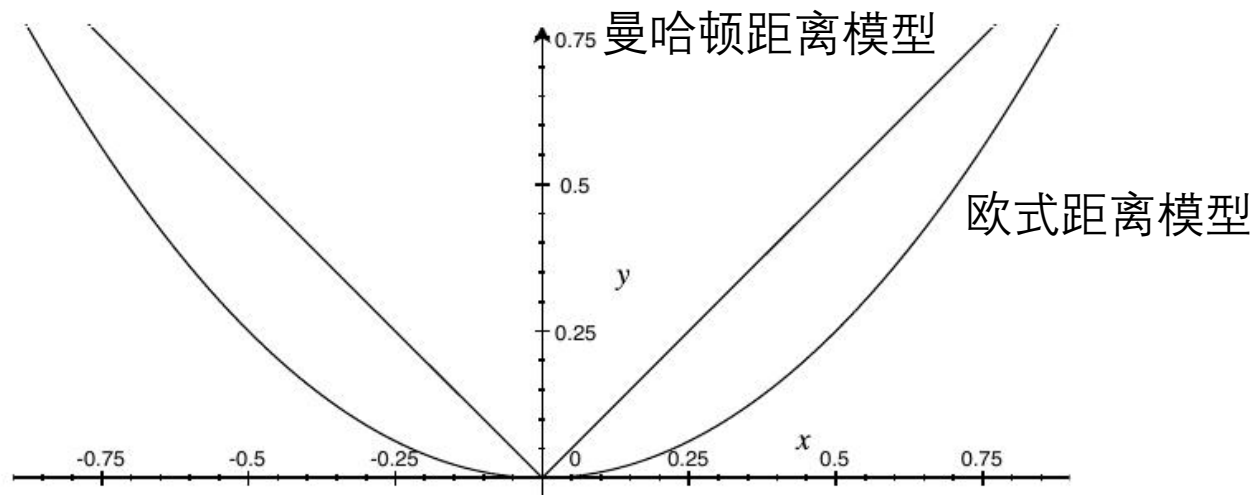
属于欧式距离模型

■ 二次型

□ 怎样计算总线长

– 对比不同的线长距离建模方式

	Steiner树	团模型		半周长	星型
模型类型	曼哈顿距离	曼哈顿距离	欧式距离	曼哈顿距离	欧式距离
准确度	★ ★ ★ ★ ★	★	★	★ ★ ★ ★	★ ★ ★
平滑度	★	★ ★	★ ★ ★ ★ ★	★ ★	★ ★ ★ ★ ★

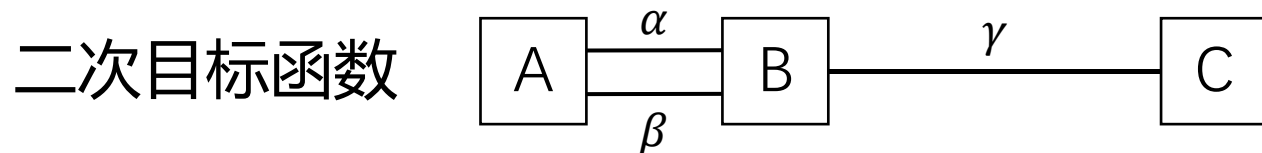
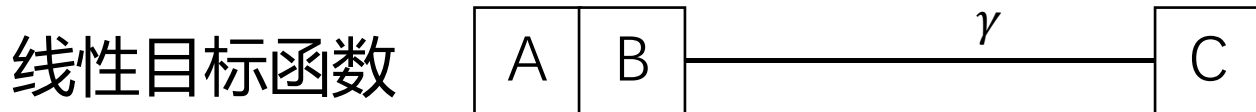
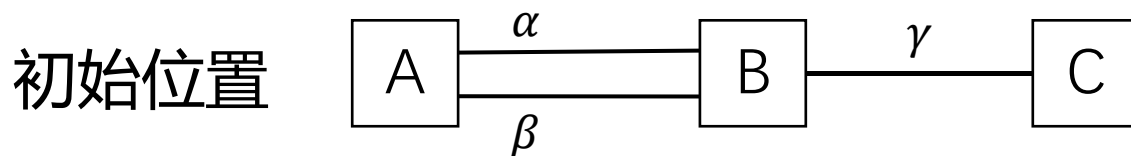


■ 二次型

□ 怎样计算总线长

– 二次方的目标函数更加容易生成均匀的布线长度

- 线性目标函数会靠近A元件与B元件，从而获得目标函数的最小值。但留下较长的 γ 线网
- 二次目标函数倾向于略微拉长 γ 线，保留一部分的 $\alpha\beta$ 线获得目标函数最小值
- 在电路设计中单一过长的导线容易引起信号完整性问题，导致设计失败



■ 二次型

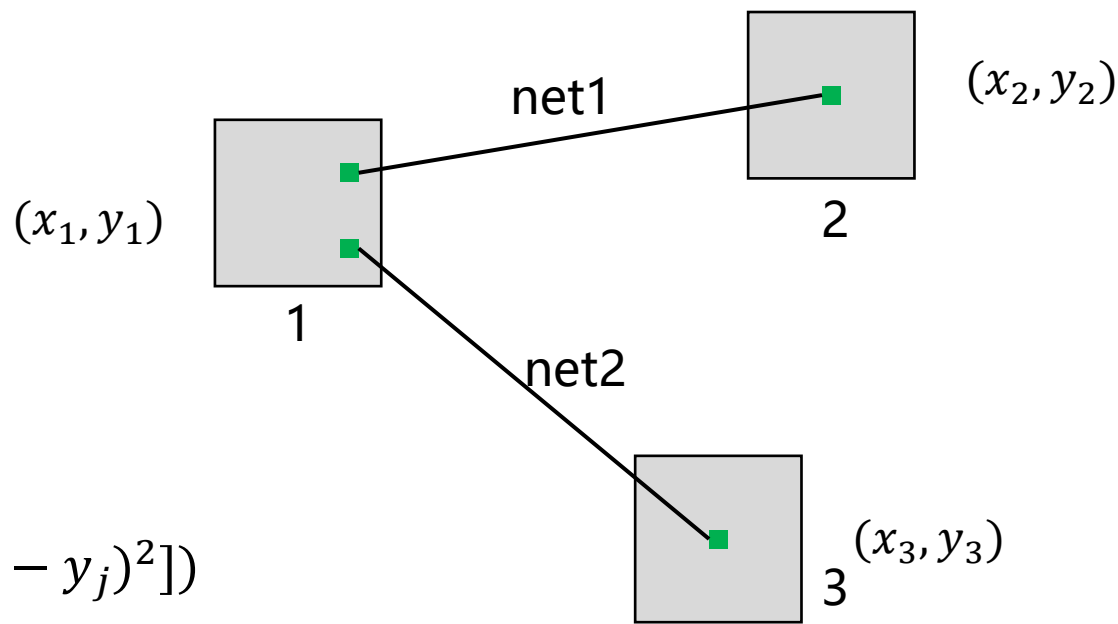
□ 怎样优化总线长

– 问题变成最小化一个二次型函数

- 计算net1和net2的线网长度
- $WL_1 = (x_1 - x_2)^2 + (y_1 - y_2)^2$
- $WL_2 = (x_1 - x_3)^2 + (y_1 - y_3)^2$
- 目标: $\min(WL_1 + WL_2)$

– 提醒

- 因为 $\min(WL) = \min(\sum [(x_i - x_j)^2 + (y_i - y_j)^2])$
 - 可以看作是 $\min(\sum (x_i - x_j)^2) + \min(\sum (y_i - y_j)^2)$, 即x方向和y方向独立
- 所以有些算法仅介绍x维度的优化



■ 二次型

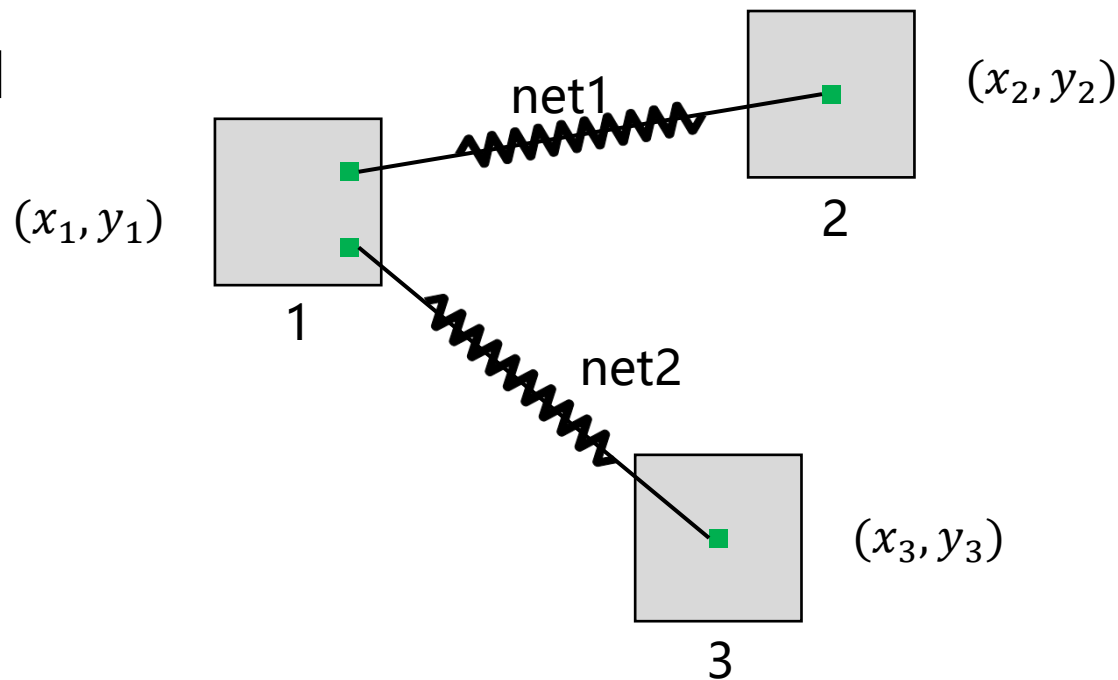
□ 怎样优化总线长

– 从物理直觉

- 把net看作是弹簧，会给模块之间一个相互吸引的力，且这个力随着距离增加而增加
- 通过仿真弹簧系统的移动得到最优布局

– 从数学直觉

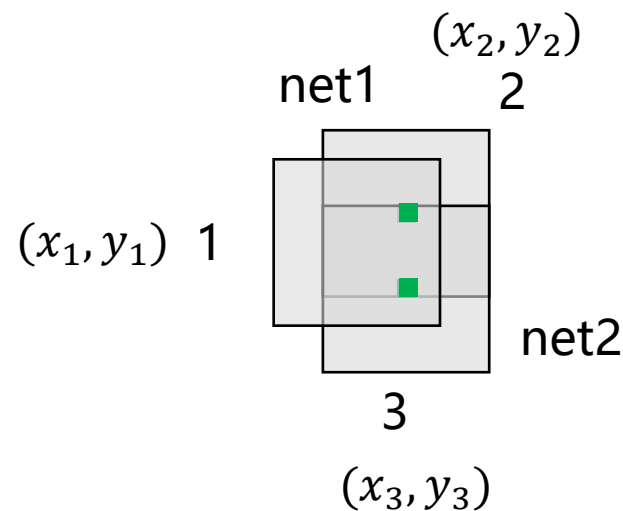
- 本质是二次规划问题
- 求 $\min_x \frac{1}{2} x^T A x - b^T x$
- 当 $Ax - b = 0$ ，即导数为0时，得到最优解



■ 二次型

□ 怎样优化线长

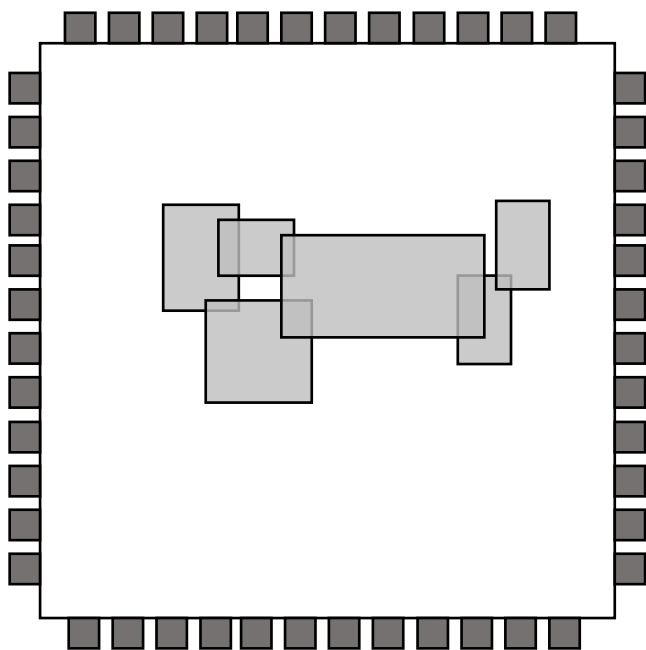
- 不加限制的解会存在一个问题
 - 无论是物理仿真还是数学解析方式
 - 最优解必然是 $x_1 = x_2 = x_3$ 且 $y_1 = y_2 = y_3$
 - 这在布局上意味着所有的模块重叠在一起
- 因此线长优化后还需要一个合法化算法
 - 把聚集的模块重新分散开，满足不重叠的约束



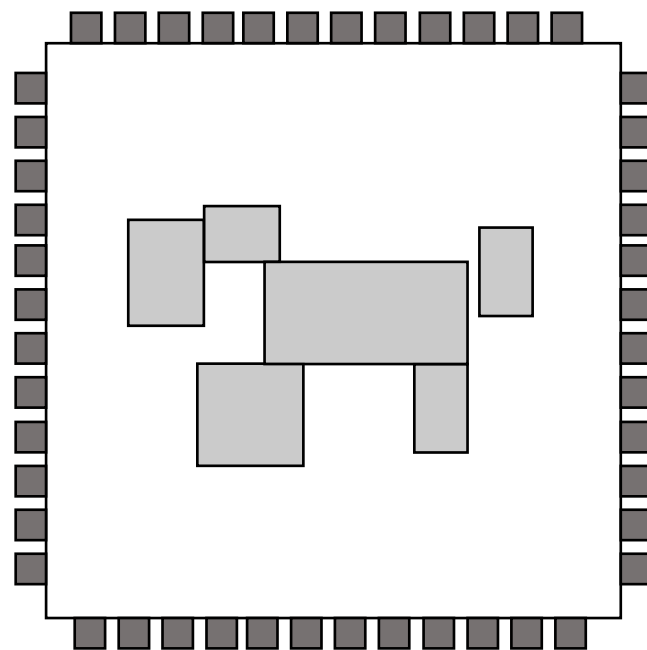
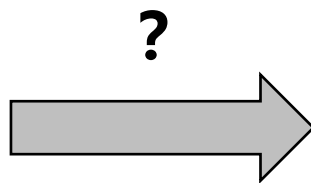
■ 二次型

□ 如何满足约束

- 合法化算法其实是另一种优化函数
 - 优化目标不是缩短线长而是减少重叠



线长优化结果

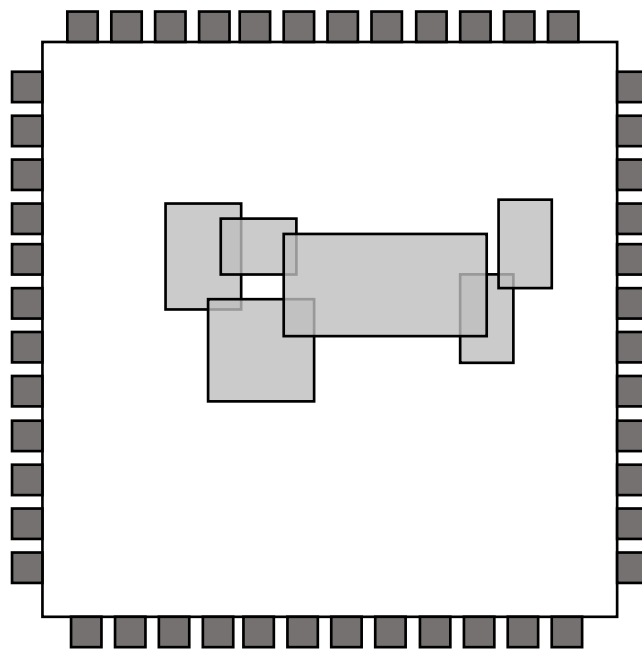


重叠优化结果

■ 二次型——SimPL

□ 粗合法化(Rough Legalization)

- 考虑水平方向的扩散
- 设计一个映射函数将芯片扩散开

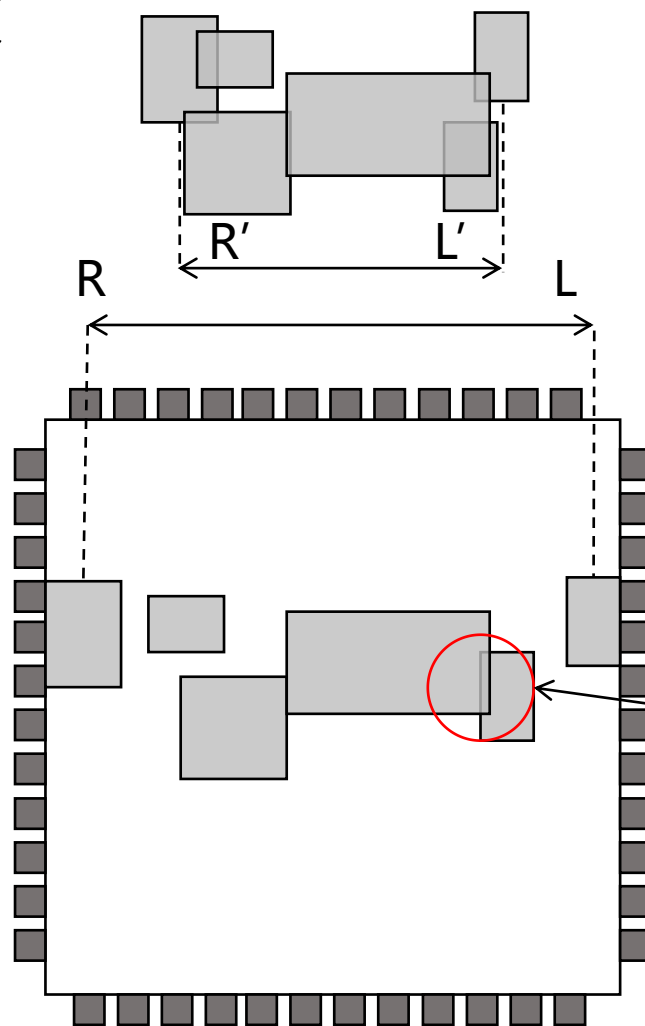


■ 二次型——SimPL

□ 粗合法化

– 例如水平线性映射函数

$$\triangleright x_i = \frac{R' - x_i'}{R' - L'} \times (R - L)$$



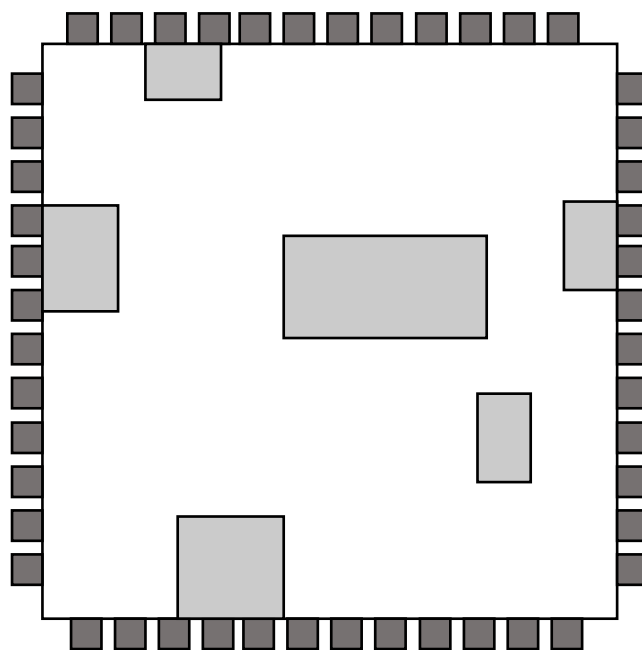
经过水平扩散后
仍有可能无法消除重叠

■ 二次型——SimPL

□ 粗合法化

– 再加一层垂直方向的线性映射函数

- 消除了布局中大部分的重叠问题
- 这样做有一个问题，扩散的目标和布局算法的目标互相冲突，算法很难收敛
 - 优化算法刚最小化总线长，聚拢一些模块，映射扩散函数就把这些模块分散出去



■ 二次型——SimPL

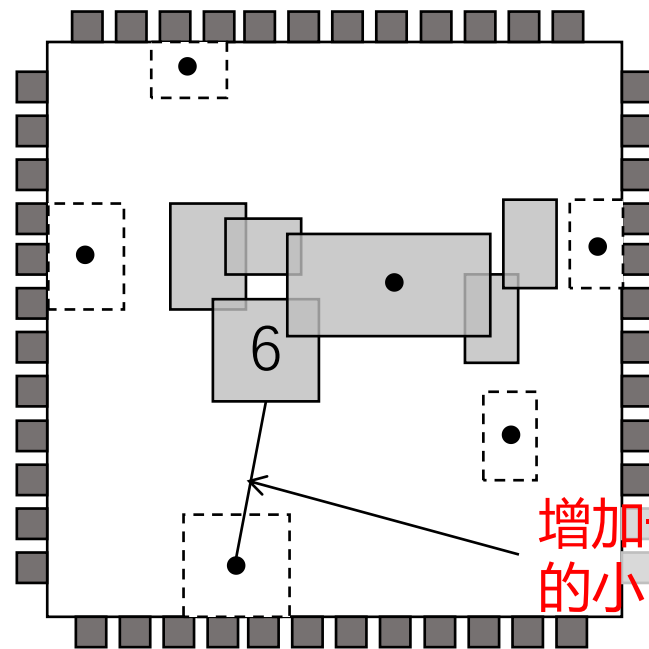
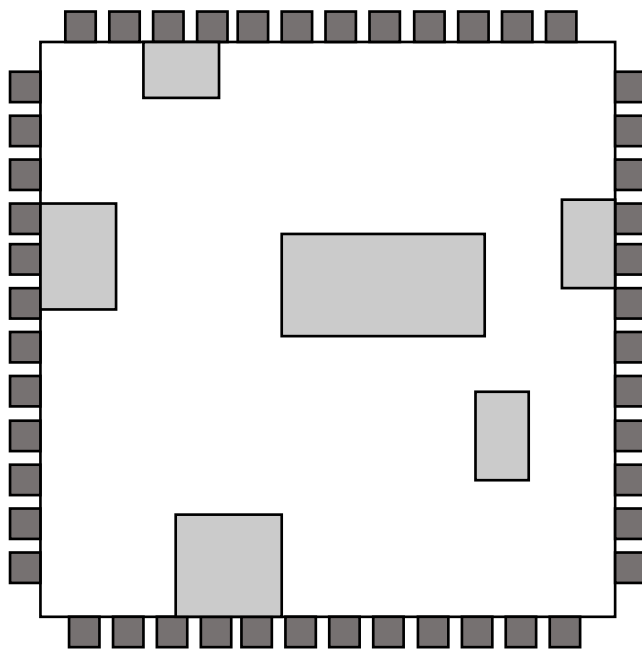
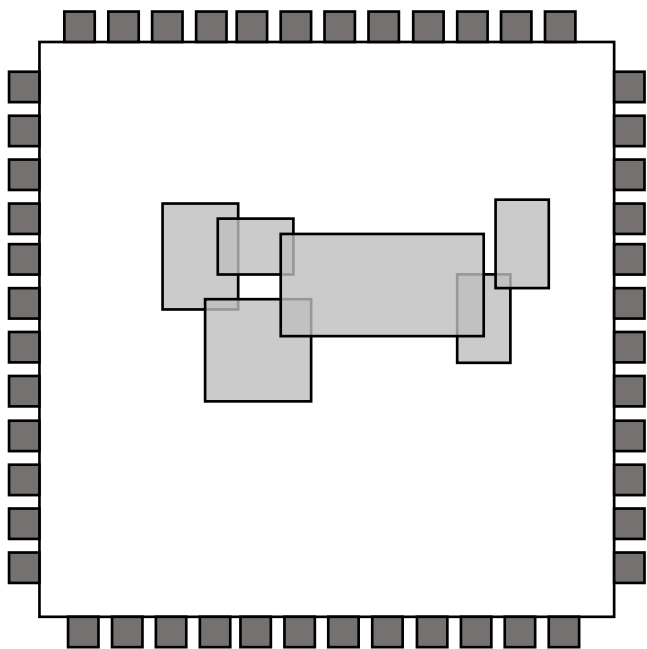
□ 粗合法化

– 修改优化算法的目标函数^[1]

➤ 使其优化线长同时考虑重叠

➤ $\min. \dots + \sum (x_6 - x_j)^2 + \dots$ **变成** $\min. \dots + \sum (x_6 - x_j)^2 + w_6(x_6 - x_{a_6})^2 + \dots$

新增虚拟节点(锚点)



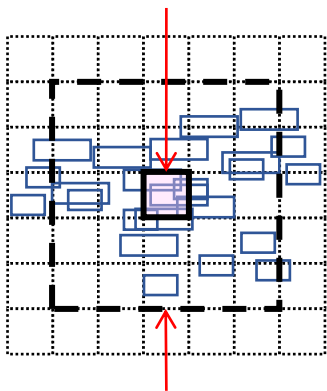
增加一个虚假的小网表

■ 二次型——SimPL

□ 粗合法化

- 需要扩散到整个芯片的范围吗?
 - 不需要，进行局部的扩散，消除模块布局的局部溢出就足够了
- SimPL筛选所有发生溢出(overflow)的区域
 - 溢出：区域内模块面积总和与区域 q 面积比值超过一个阈值 λ (设定的布局密度上限)
 - 在 q 周围找到一个最小的区域 B ，使得扩散后整个区域 B 不会发生溢出

发生溢出的区域 q

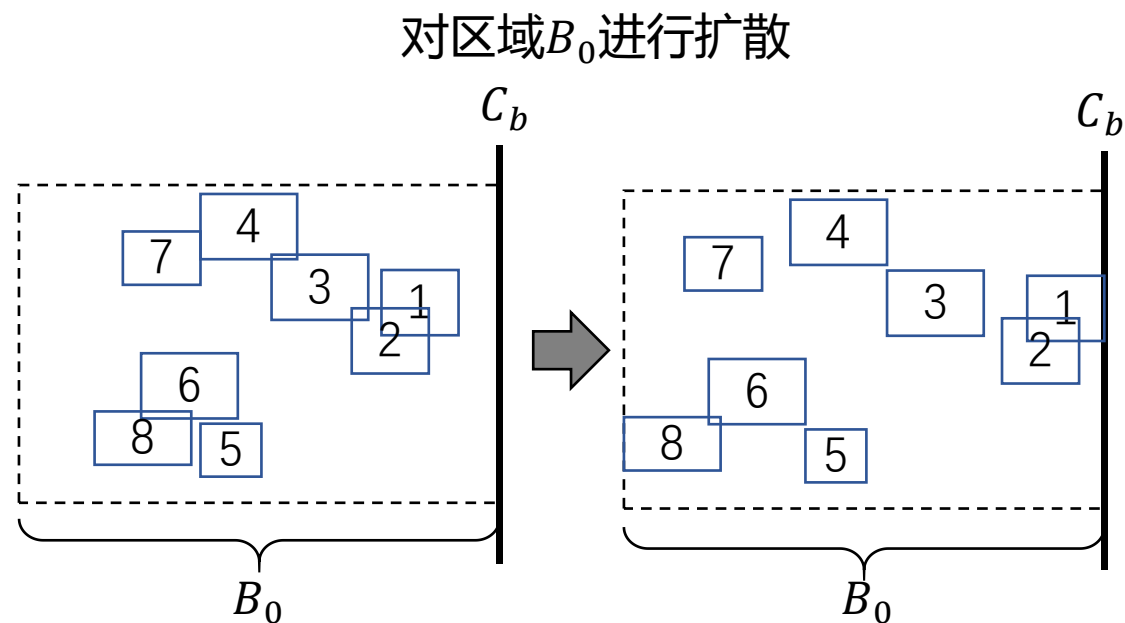
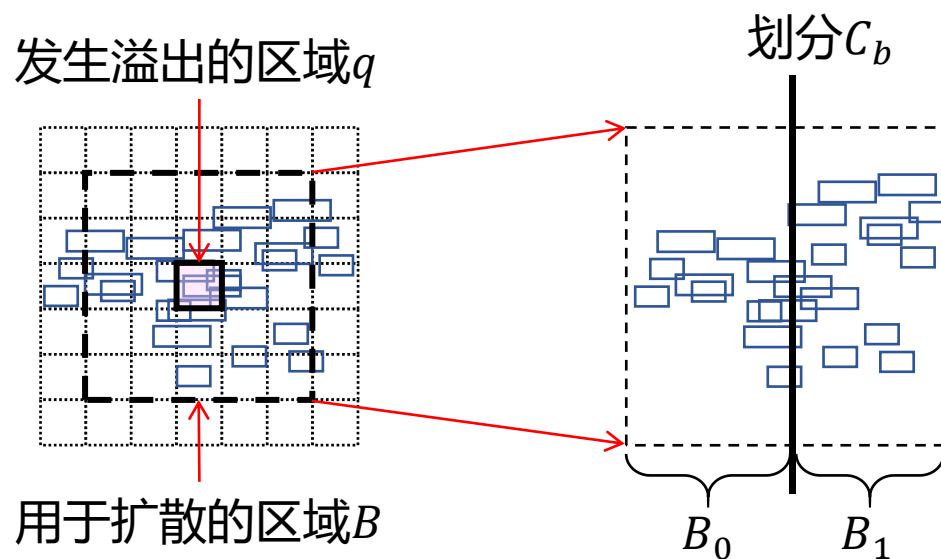


用于扩散的区域 B

■ 二次型——SimPL

□ 粗合法化

- 在扩散区域 B , 找到一个划分 C_b
 - 使得区域 B_0 和 B_1 的模块密度基本相等
- 对区域 B_0 和 B_1 分别进行类线性的扩散
 - SimPL对一些本来就靠的近的模块进行打包组合, 一同扩散
 - 所以SimPL称之为类线性的扩散



■ 二次型——SimPL

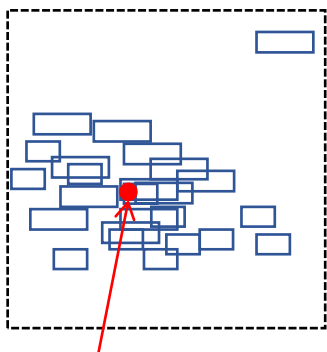
□ 粗合法化

– 先划分后扩散

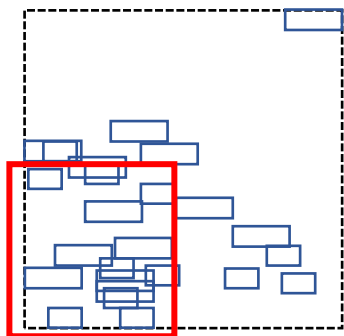
➤ 避免局部的拥塞，适应模块密度存在偏移的情况

– 不划分直接扩散可看作是相对区域中心进行扩散

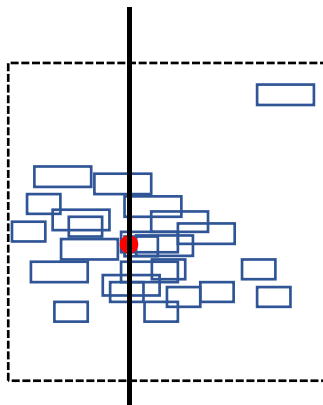
➤ 对下图进行扩散并不能改善溢出



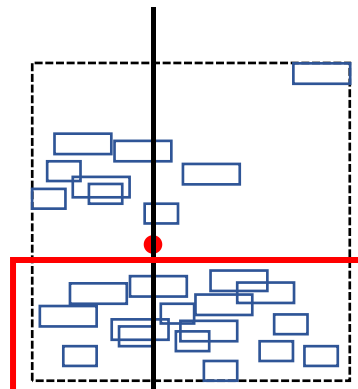
模块密度中心偏向左下角



大部分模块聚集在左下角
仍然有可能发生溢出



按照密度划分进行扩散



考虑区域模块密度，设定划分边界
可以把更多模块向右边扩散

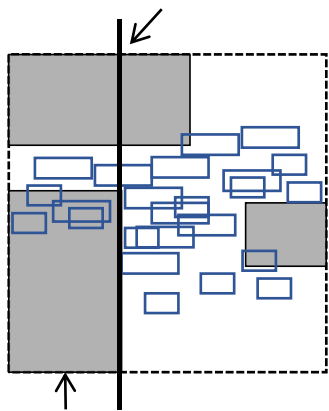
■ 二次型——SimPL

□ 粗合法化

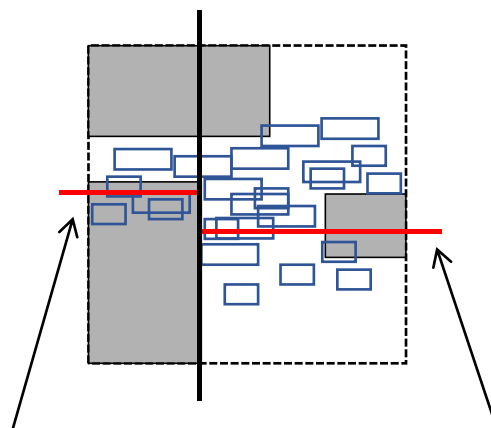
– 扩散的方向更有目的性

- 而且可以考虑到已经放置宏模块的情况
- 对于布置宏模块的区域，相当于大部分面积已经被宏模块使用，那么在扩散普通模块时应该尽量避免向宏模块所在区域进行扩散
- 若扩散效果不能消除重叠，可以进行追加

划分边界设置在这，防止继续向宏模块扩散

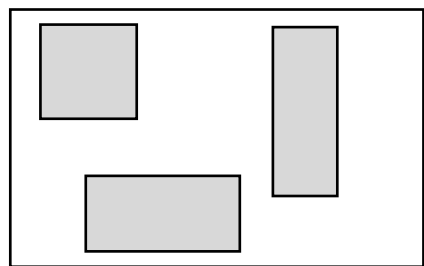


灰色的宏模块占据区域大部分面积

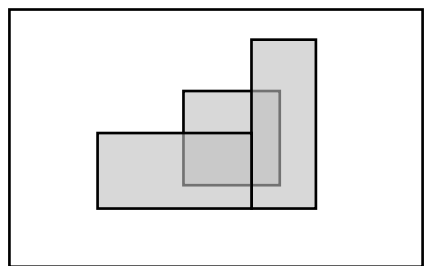


可以进一步划分增强扩散效果

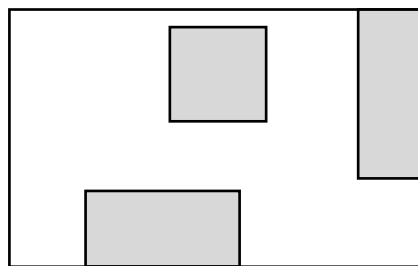
■ 二次型——SimPL



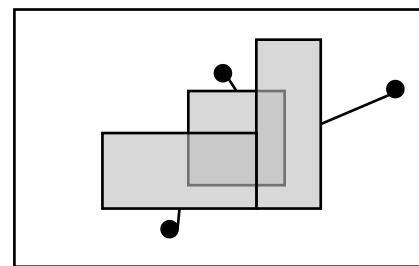
初始布局



优化线长

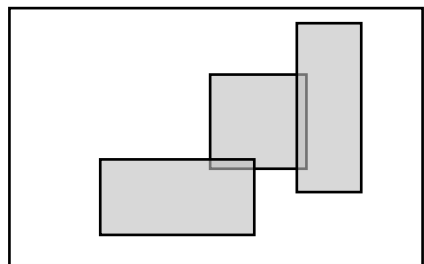


合法化

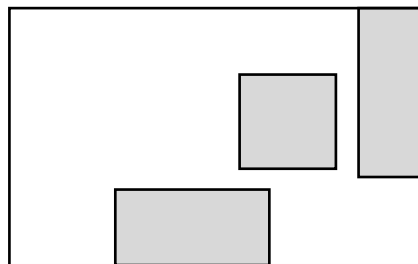


插入锚点

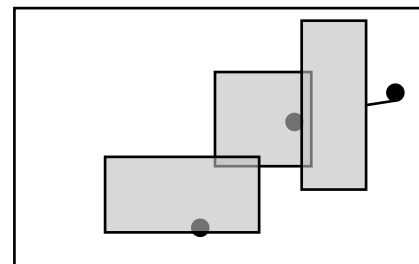
第一轮迭代



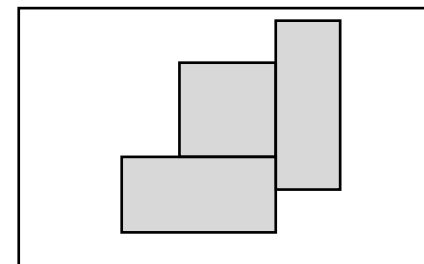
优化线长



合法化



插入锚点



“最优解”

第二轮迭代

■ 二次型——SimPL

□ SimPL 布局算法总结

– 迭代优化

- 布局结果的线长在上界和下界之间不断跳动，最后收敛于一个稳定的值

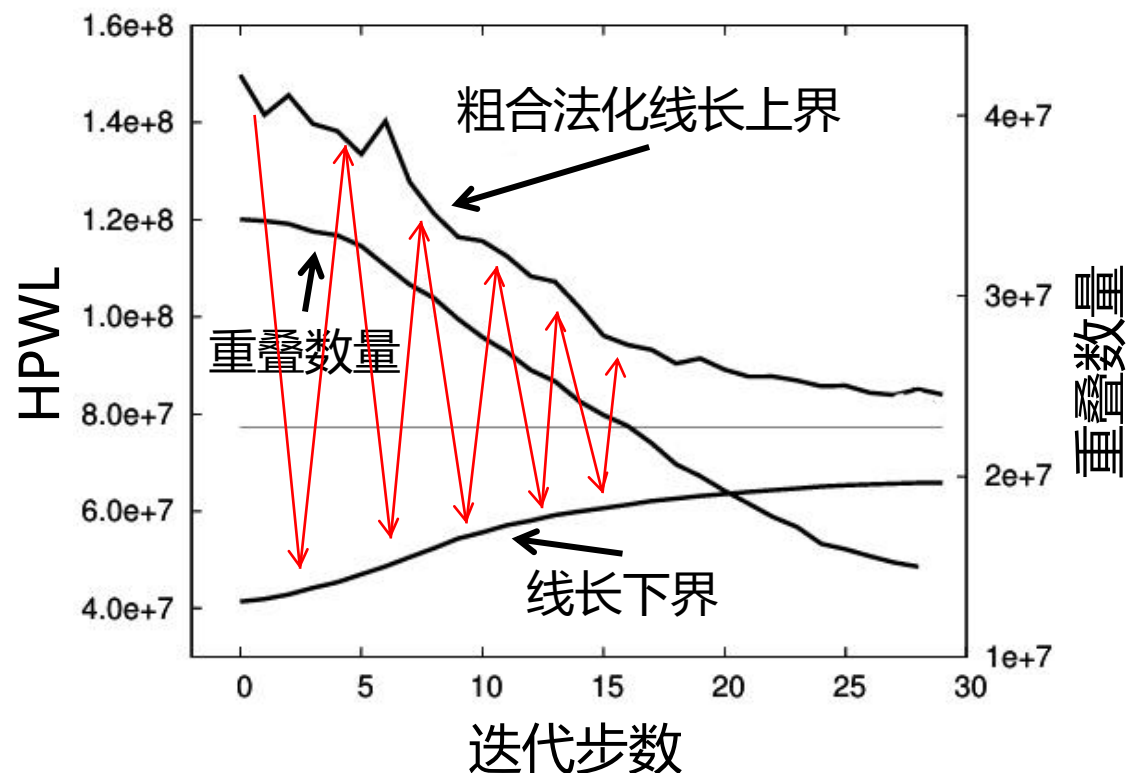
– 粗合法化

- 插入虚拟锚点，修改布局优化算法的目标函数，使得算法优化线长的同时也能考虑重叠约束

- 计算效率高

– 不足

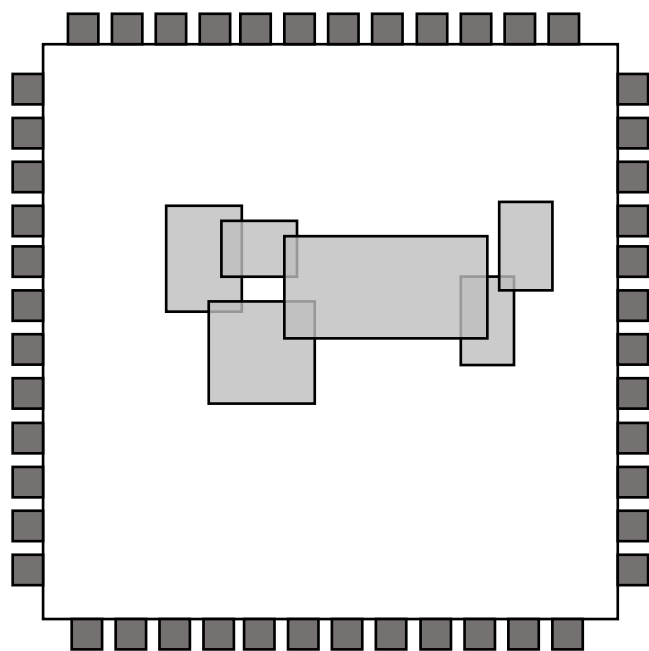
- 类线性的扩散方法，不一定适合复杂的布局问题



■ 二次型——SimPL

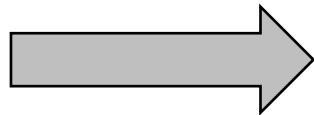
□ 课后思考

- 在什么情况下线性映射的效果会比较差
- 如何设计一种更好的映射方法

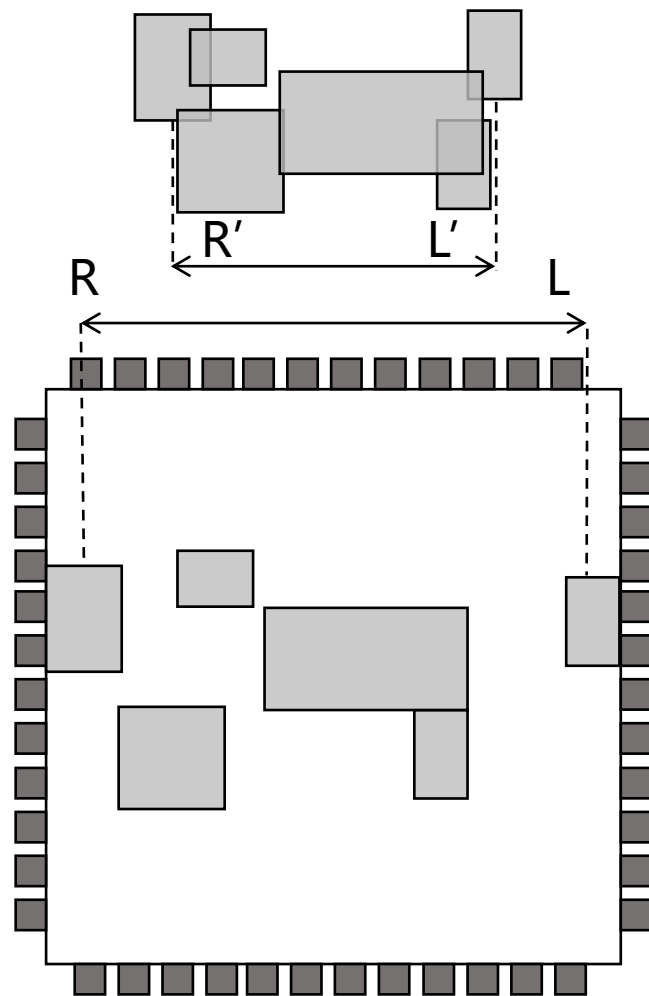


输入

线性映射



$$x_i = \frac{R' - x_i'}{R' - L'} \times (R - L)$$



输出

■ 二次型——Gordian

□ 问题定义

– 输入

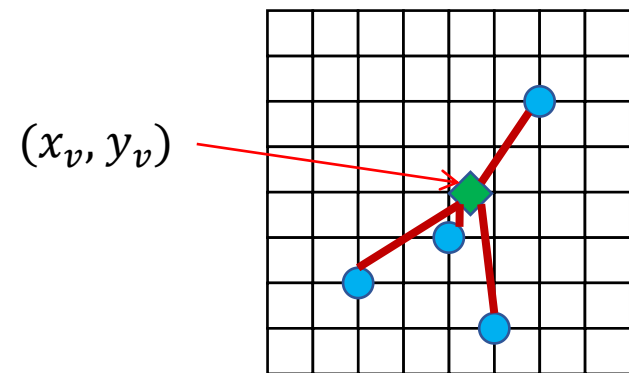
- 模块集合 V 和线网集合 E

– 输出

- 每个模块的坐标位置 (x, y)
- 给定布局区域内没有模块重叠

– 优化目标

- 最小化所有线网的长度之和 $\Phi = \frac{1}{2} \sum_{v \in E} L_v w_v$, 其中 w_v 是线网 v 的权重
- $L_v = \sum_{\mu \in v} [(x_\mu - x_v)^2 + (y_\mu - y_v)^2]$, 其中 x_v, y_v 表示线网的中心点坐标, x_μ, y_μ 是与线网 v 相连的模块坐标 (即星型线长预估模型)
- 尽可能聚集相同线网的模块, 同时保证高权重的线网优先优化



星型线长预估模型

■ 二次型——Gordian

□ 算法流程

- 使用数学方法优化目标函数
 - $\Phi = \frac{1}{2} \sum_{v \in E} L_v w_v$ 这个二次优化问题
 - 最小化上述目标函数
- 使用划分算法消除重叠
 - 划分模块到不同的芯片区域
 - 限制模块的可布局区域从而缓解模块的重叠
- 迭代以上两步直到划分消除大部分重叠

■ 二次型——Gordian

□ 算法流程

- Gordian方法不使用划分策略来减少问题规模
- 而是利用划分来限制模块的可移动范围
 - 每次利用数学方法优化目标函数时都是计算整个网表，而不是划分后的小网表
 - 为防止模块聚集在划分边界，给优化目标添加一个“重心”约束
- 因为划分质量的好坏会直接影响最终布局结果的质量
 - Gordian细致实现优化的划分算法

■ 二次型——Gordian

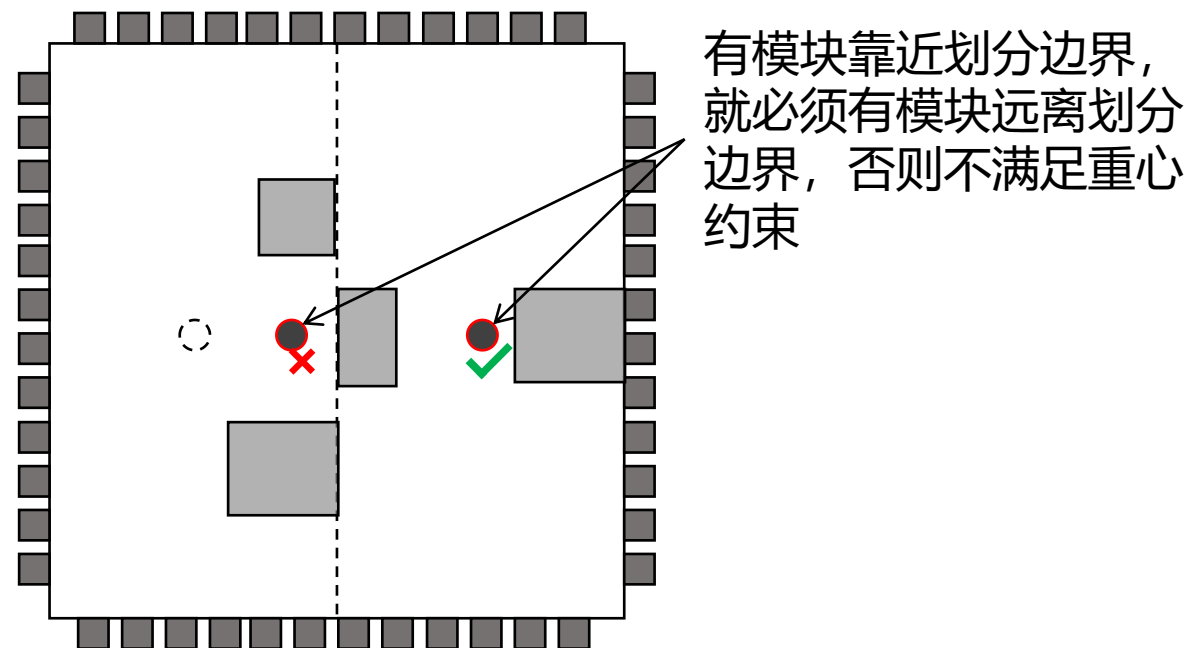
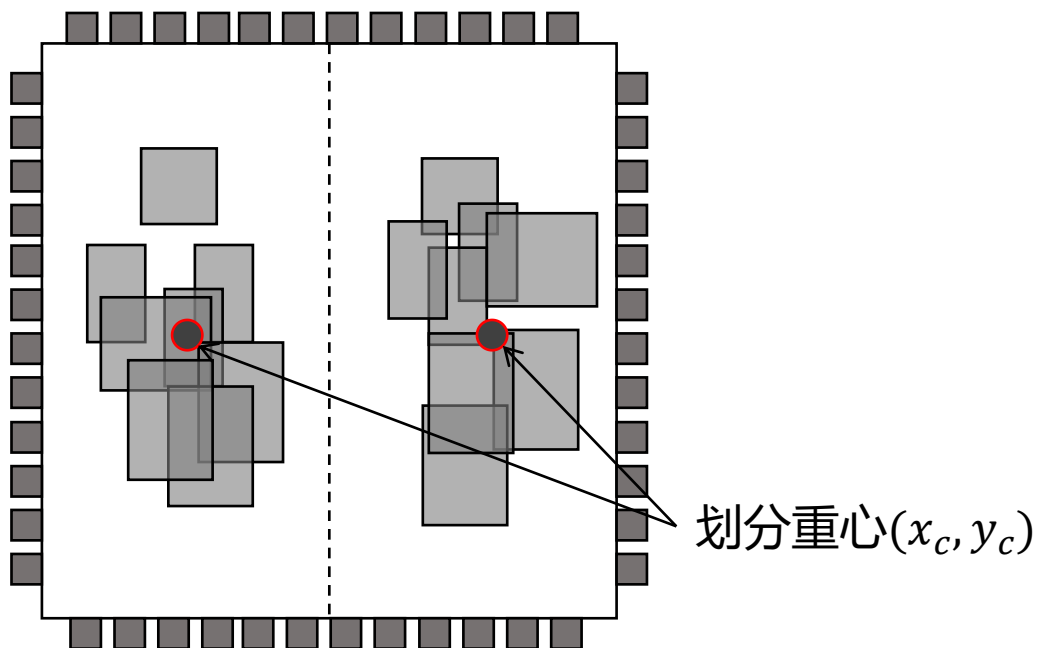
□ “重心” 约束

– 约束: $\sum_{u \in M_p} w_u x_u = x_c, \sum_{u \in M_p} w_u y_u = y_c$

➤ 其中 w_u 是模块 u 的权重, M_p 表示属于划分 p 的模块集合

➤ (x_c, y_c) 一般情况下是布局区域的中心

– 这样划分后的模块就不会聚集在上次划分边界附近, 提高划分的质量



■ 二次型——Gordian

□ 优化划分算法

– 初划分，快速生成初始解

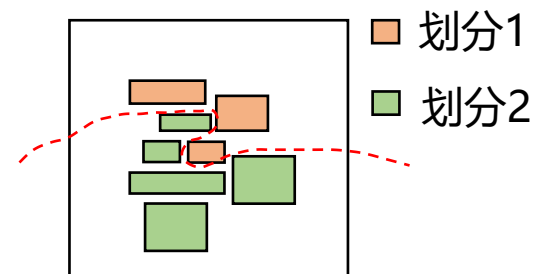
- 约束: $\frac{F_p}{F_{p'}} = \alpha$, 即划分 p, p' 包含的模块面积之和比例接近 α
- 注意, 此时不考虑割代价

– 划分质量提高

- 使用Fiduccia-Mattheyses算法提高划分质量, 最小化割代价

– 再划分(Repartitioning)

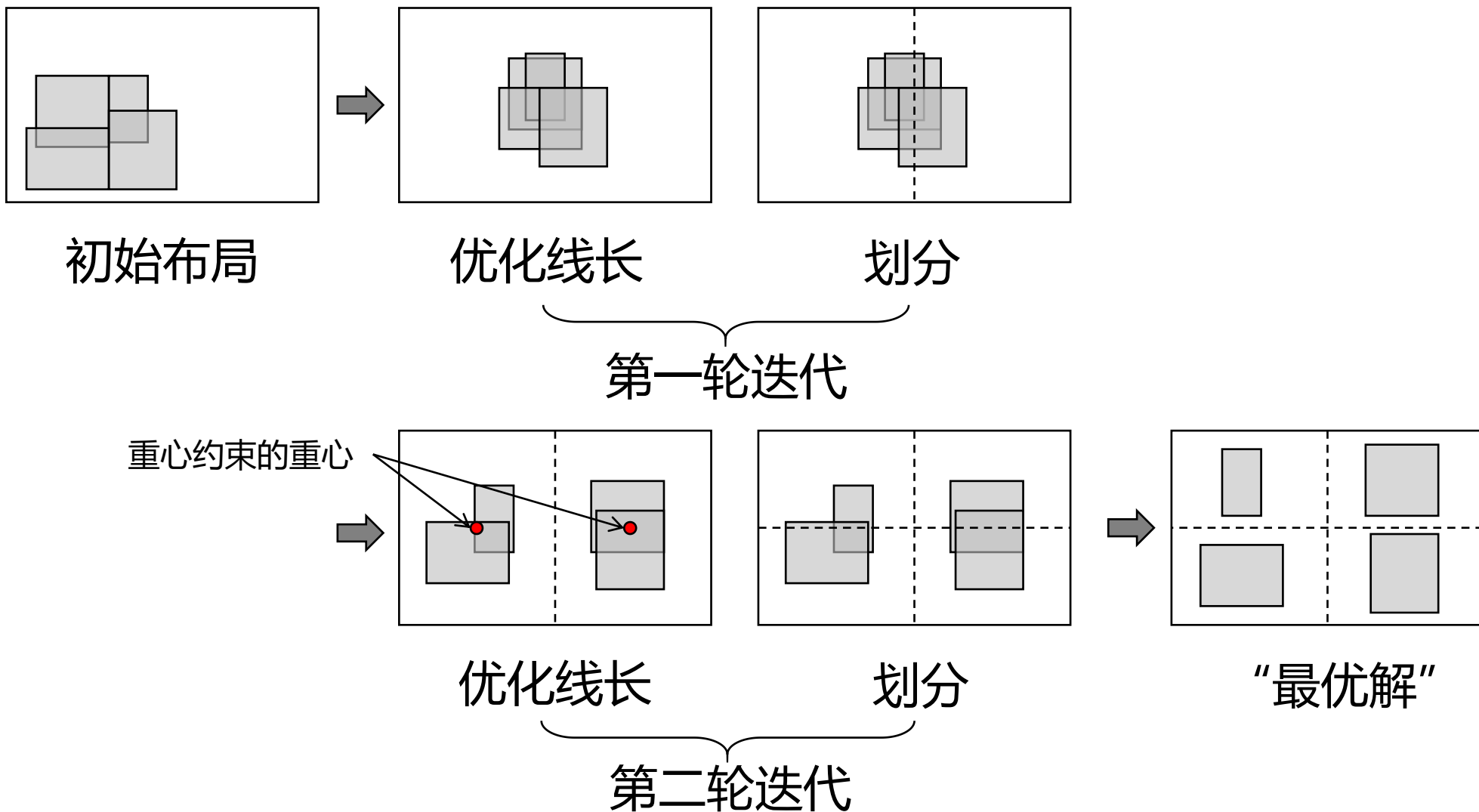
- 减少因为不合理初划分牺牲解质量情况
 - 尤其是当划分线靠近模块密度高的中心, 这导致模块分配的子集呈现随意性
- 具体做法是合并两个子集, 重新进行初划分和FM算法优化
- 对于两个子集 X, X' , 如果 $\exists x_{\mu'} > \min_{\mu'' \in X'} \{x_{\mu''}\}, x_{\mu'} \in X$ (存在集合间交错现象) 就触发再划分, 改进划分结果



有橙色划分的模块深入绿色划分区域, 存在交错现象。需要再次划分

■ 二次型——Gordian

□ 算法迭代流程总览



■ 二次型

□ 总结

– 概括

- 把布局问题建模为求解大型稀疏的线性方程组问题，进而最小化整个线长
- 为缓解模块重叠问题，不约而同选择添加分散的锚定点、连接边和划分等方式，把模块从高密度区域拖到低密度区域

– 优势

- 非常高效，适合解决大规模布局问题
- 二次方的线长预估模型的可微性，使得利用梯度下降法高效求解最小值成为可能

– 不足

- 在解质量和鲁棒性上稍逊非线性布局器

② 非线性函数优化布局

■ 非线性

1970-1980	1980-1990	1990-2010			>2010	
基于划分	模拟退火	最小割 (多层次)	解析式		解析式	
			二次型	非线性	二次型	非线性
Breuer	Timberwolf VPR	FengShui	GORDIAN	APlace	POLAR	ePlace Replace
Dunlop & Kernighan	Dragon	Capo	BonnPlace	Naylor Synopsis	SimPL ComPLx	DREAMPlace
Quadratic Assignment		Capo+Rooster	mFar	NTUplace	MAPLE	
Cadence QPlace			Kraftwerk	mPL6		
			FastPlace			
			Warp3			

■ 非线性——NTUplace

- ❑ Tung-Chieh Chen, Tien-Chang Hsu, Zhe-Wei Jiang, Yao-Wen Chang, [“NTUplace: a ration partitioning based placement algorithm for large-scale mixed-size designs,”](#) ISPD, 2005.
- ❑ Tung-Chieh Chen, Zhe-Wei Jiang, Tien-Chang Hsu, Hsin-Chen Chen, Yao-Wen Chang, [“NTUplace3: An Analytical Placer for Large-Scale Mixed-Size Designs With Preplaced Blocks and Density Constraints,”](#) TCAD, 2008.
- ❑ Meng-Kai Hsu, Yi-Fang Chen, Chau-Chin Huang, et al. [“NTUplace4h: A Novel Routability-Driven Placement Algorithm for Hierarchical Mixed-Size Circuit Designs,”](#) TCAD, 2014.
- ❑ Chau-Chin Huang, Hsin-Ying Lee, Bo-Qiao Lin, et al. [“NTUplace4dr: A Detailed-Routing-Driven Placer for Mixed-Size Circuit Designs With Technology and Region Constraints,”](#) TCAD, 2017.

■ 非线性——NTUplace

□ 问题定义

– 输入

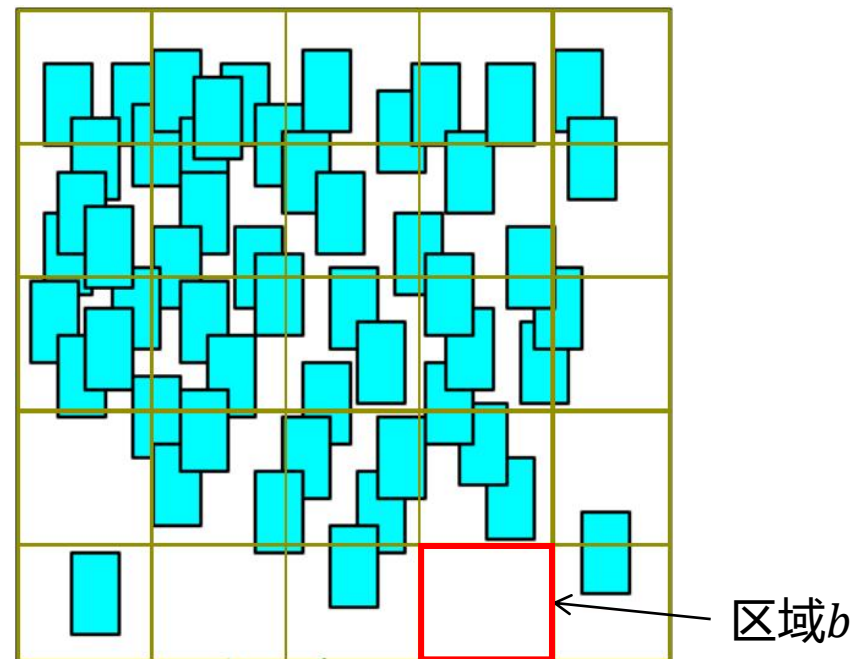
- 模块集合 V 和线网集合 E

– 输出

- 每个模块的坐标位置 (x, y)
- 给定布局区域内没有模块发生重叠

– 优化目标

- $\min_s WL + \lambda D(x, y)$, s 表示一种布局结果
- 其中 WL 是线长预估函数, $D(x, y)$ 则是 (x, y) 区域和密度相关惩罚函数



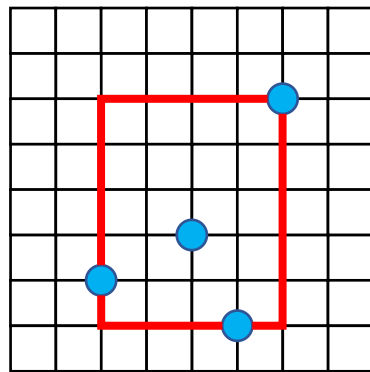
■ 非线性——NTUplace

□ 平滑目标函数WL

$$- WL(e) = \sum_{e \in E} \left(\max_{v_i, v_j \in e} |x_i - x_j| + \max_{v_i, v_j \in e} |y_i - y_j| \right)$$

➤ HPWL线长模型

➤ 这个线长模型不光滑且非凸，不能直接利用梯度方法求解



– 使用LSE(log-sum-exp)进行平滑

➤ 令 $LSE(x: \gamma) = \gamma \ln \sum_i e^{x_i/\gamma}$

➤ 利用LSE的性质 $\max\{x_1, \dots, x_n\} < LSE(x: \gamma) \leq \max\{x_1, \dots, x_n\} + \gamma \ln n$

➤ 易得证 $LSE(x: \gamma) \approx \max\{x_1, \dots, x_n\}$ 而 $-LSE(x: -\gamma) \approx \min\{x_1, \dots, x_n\}$

➤ 得到WL新的光滑的表示函数：

$$\bullet \quad WL(x, y) = \gamma \left(\underbrace{\ln \sum_{v_i \in e} e^{x_i/\gamma}}_{\max_{v_i, v_j \in e} |x_i - x_j|} + \underbrace{\ln \sum_{v_i \in e} e^{-x_i/\gamma}}_{\max_{v_i, v_j \in e} |x_i - x_j|} + \underbrace{\ln \sum_{v_i \in e} e^{y_i/\gamma}}_{\max_{v_i, v_j \in e} |y_i - y_j|} + \underbrace{\ln \sum_{v_i \in e} e^{-y_i/\gamma}}_{\max_{v_i, v_j \in e} |y_i - y_j|} \right)$$

■ 非线性——NTUplace

□ LSE的特性

$$- \max\{x_1, \dots, x_n\} < LSE(x) \leq \max\{x_1, \dots, x_n\} + \log n$$

$$\begin{aligned} \max\{x_1, \dots, x_n\} &= \log(\exp(\max x_i)) \\ &< \log(\exp(x_1) + \dots + \exp(x_n)) \\ &\leq \log(n \times \exp(\max(x_i))) \\ &= \max\{x_1, \dots, x_n\} + \log n \end{aligned}$$

➤ 因此可以说 $LSE(x: \gamma) \approx \max\{x_1, \dots, x_n\}$, 误差最大是 $\log n$

$$- \text{而} -LSE(x: -\gamma) \approx \min\{x_1, \dots, x_n\}, \text{ 假设 } x_m = \min\{x_1, \dots, x_n\}$$

➤ 因为 $-LSE(x: -\gamma) = -(-\gamma \ln \sum_{v_i \in e} e^{x_i / -\gamma}) = \gamma \ln \sum_{v_i \in e} e^{-x_i / \gamma}$

➤ 恰好对指数取反, 于是最小值 x_m 变成了数组的最大值

➤ 近似替代 $\min$ 函数

□ 其他平滑工作可以参考[BiG: Bivariant smoothing^{\[1\]}](#)

[1] Fan-Keng Sun, Yao-Wen Chang, "Big: A Bivariate Gradient-Based Wirelength Model for Analytical Circuit Placement." DAC, 2019.

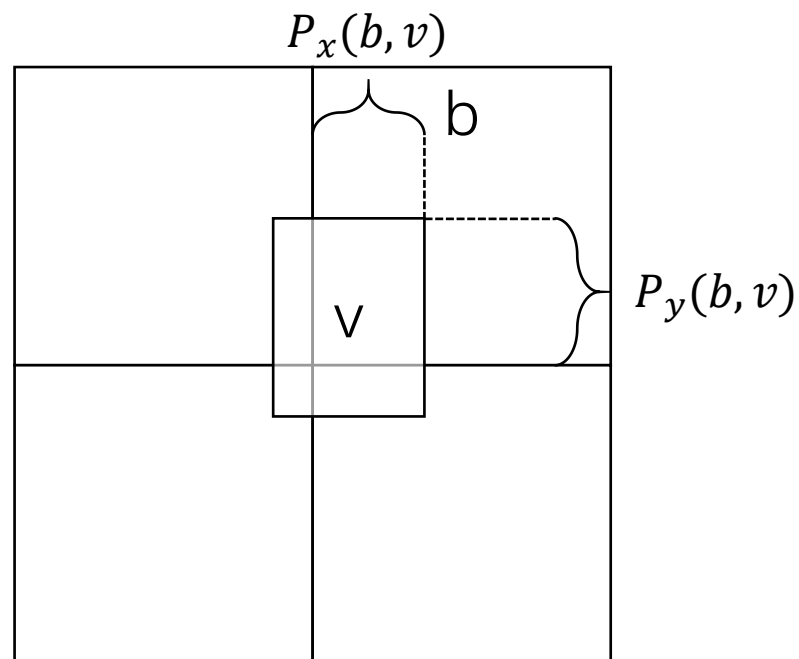
■ 非线性——NTUplace

□ 平滑密度计算 $d_b(x, y)$

– $d_b(x, y) = \sum_{v \in V} P_x(b, v) P_y(b, v),$

➤ 其中 P_x 代表 x 方向的重叠量, P_y 是 y 方向的重叠量, b 表示一个区域, v 表示模块

– 直观理解就是模块 v 和区域 b 重叠部分的面积



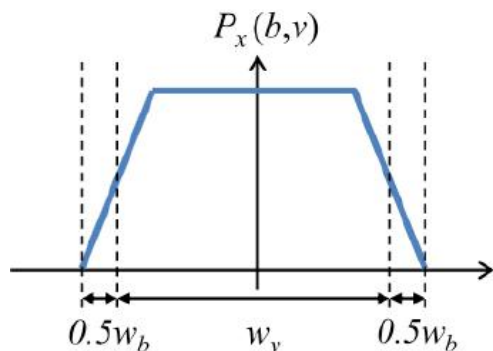
■ 非线性——NTUplace

□ 平滑密度计算 $d(x, y)$

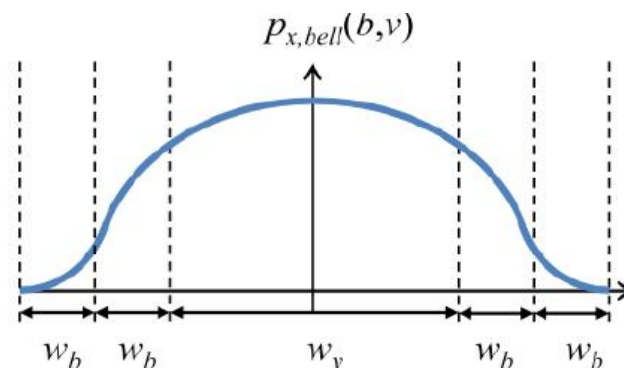
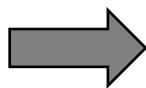
– 钟形曲线平滑，为了便于利用梯度法求解目标函数

$$\text{➤ } P_x(b, v) = \begin{cases} 1 - ad_x^2 & , 0 \leq d_x \leq \frac{w_v}{2} + w_b \\ b(d_x - \frac{w_v}{2} - 2w_b)^2 & , \frac{w_v}{2} + w_b \leq d_x \leq \frac{w_v}{2} + 2w_b \\ 0 & , \frac{w_v}{2} + 2w_b \leq d_x \end{cases}$$

➤ 其中 d_x 代表模块 v 和区域 b 的在 x 方向上的中心到中心距离(center-to-center distance), w_b 和 w_v 分别代表区域的宽度和模块的宽度



不平滑、非凸函数



平滑、凸函数

■ 非线性——NTUplace

□ 算法流程

- 全局布局(Global Placement)
 - 和二次型类似，采用迭代优化方法
 - 采用共轭梯度法优化目标函数
- 合法化布局(Legalization)
 - 按照优先级，采用贪心策略布局模块
- 详细布局(Detailed Placement)
 - 滑动窗口，局部交换以优化线长，局部扩散降低模块密度

■ 非线性——NTUplace

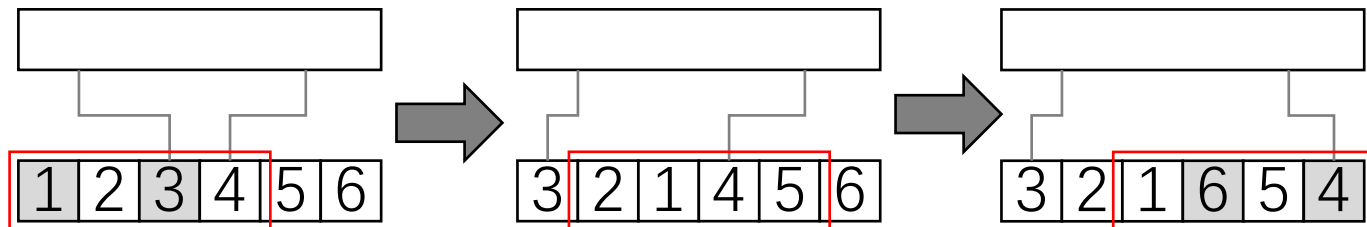
□ 合法化布局

- 计算模块的优先级，按照优先级高低依次进行布局合法化
 - 优先级计算公式: $priority(v_i) = k_1x_i + k_2h_i + k_3w_i$, 其中 $k_1 = 1000$, 而 $k_2 = k_3 = 1$
 - 目的是先按 x 方向依次优化模块布局, 如果 x 相同, 则面积大的模块优先
- 布局移动策略是贪心策略
 - 找到最近的可行布局区域, 然后进行布局

■ 非线性——NTUplace

□ 详细布局

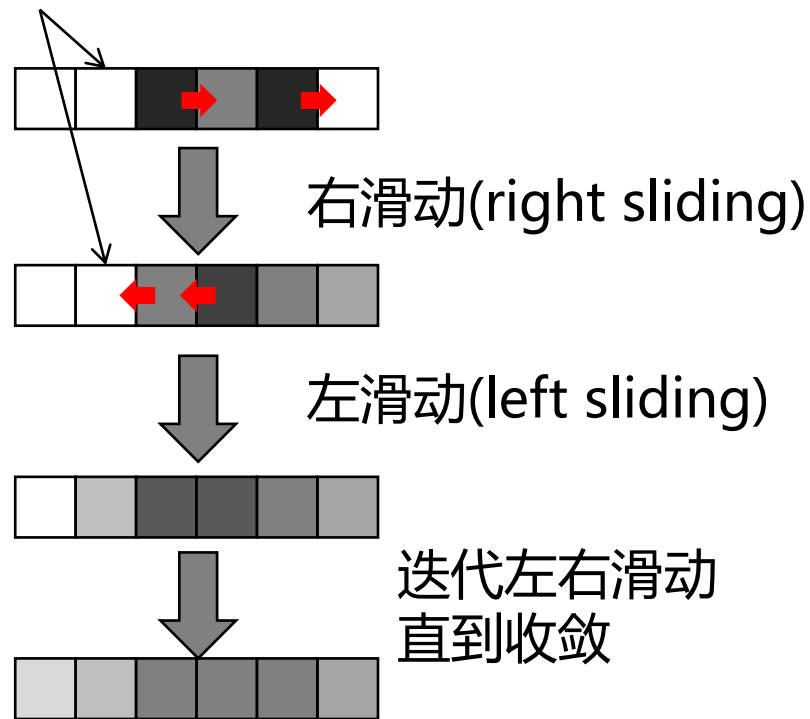
- 采用滑动窗口进行优化
 - 主要目标是模块密度
 - 次要目标是线长
- 交换和移动仅考虑窗口内的模块
 - 线长优化进行窗口内模块的交换
 - 模块密度优化时，计算模块密度差，仅允许模块从高密度区域向低密度区域流动
 - 二者同时进行，直到不再产生线长或密度的收益



线长优化

滑动窗口

注意仅向一个方向流动



■ 非线性——NTUplace

□ 总结

– 不足之处

- 复杂的模块密度计算公式
- 模块移动只能考虑局部的密度信息，没有全局信息

■ 非线性——ePlace

- ❑ Jingwei Lu, Pengwen Cheng, Chin-Chih Chang, et al, “[ePlace: Electrostatics-Based Placement Using Fast Fourier Transform and Nesterov’s Method](#),” TODAES, 2015.
- ❑ Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, et al “[RePlAce: Advancing Solution Quality and Routability Validation in Global Placement](#),” TCAD, 2018.
- ❑ Yibo Lin, Zixuan Jiang, Jiaqi Gu, “[DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement](#),” TCAD, 2020. (DAC 2019 Best Paper Award)

■ 非线性——ePlace

□ 问题定义

— 输入

- 模块集合 V 和线网集合 E

— 输出

- 每个模块的坐标位置 (x, y)
- 给定布局区域内没有模块发生重叠

— 优化目标（目标和NTUplace基本一致）

- $\min_{\mathcal{S}} WL + \lambda N(\mathcal{S})$ 是最终目标函数， $\mathcal{S}$ 表示一种布局结果
- 其中 WL 是线长预估函数， $N(\mathcal{S})$ 则是和密度相关的惩罚函数

■ 非线性——ePlace

□ $\min_{\mathcal{S}} WL + \lambda N(\mathcal{S})$

- 如何平滑 WL , ePlace采用加权平均(**W**ighted **A**verage)的线长模型

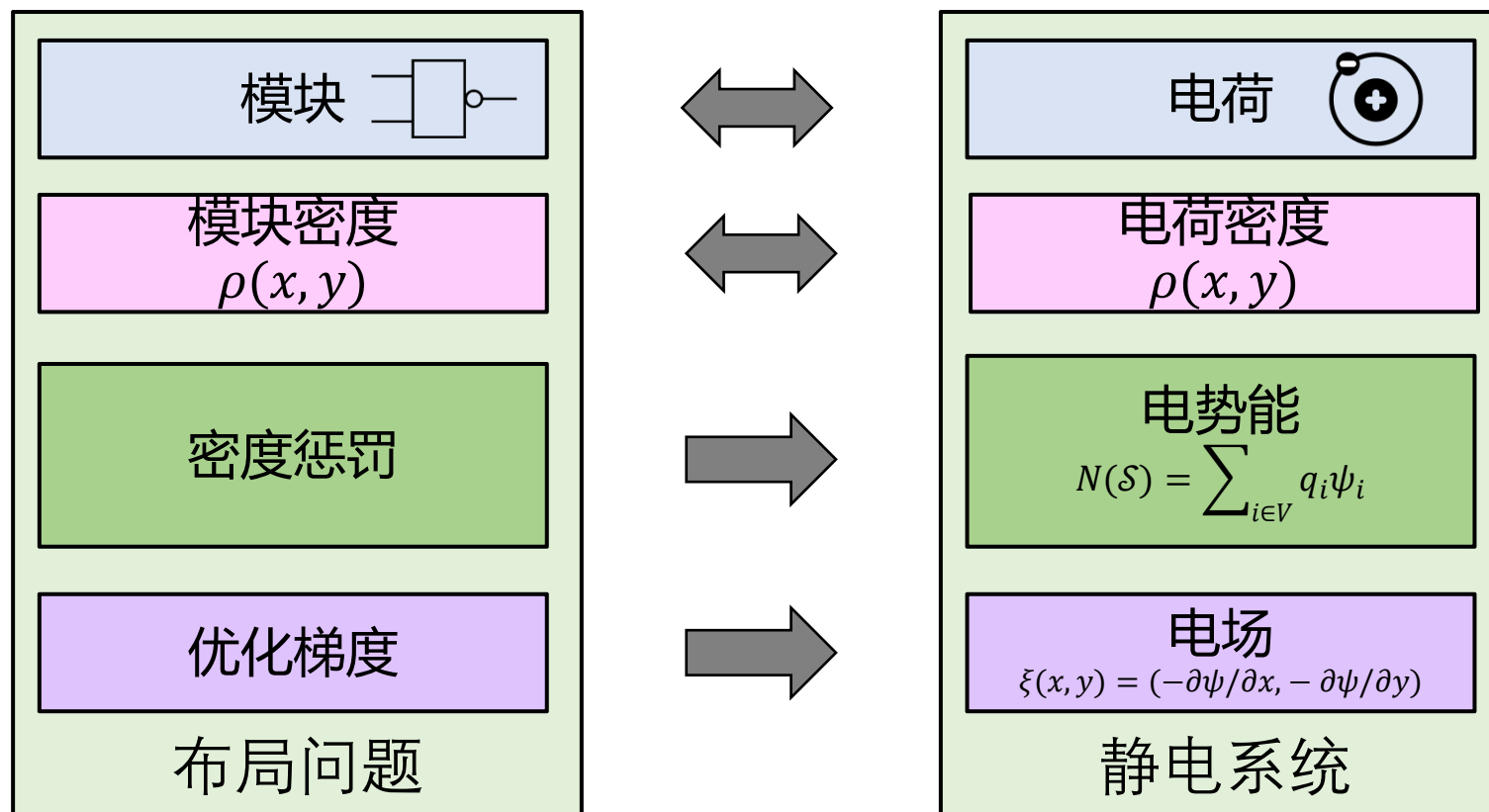
$$WL_e = \left(\frac{\sum_{i \in e} x_i \exp(x_i/\gamma)}{\sum_{i \in e} \exp(x_i/\gamma)} - \frac{\sum_{i \in e} x_i \exp(-x_i/\gamma)}{\sum_{i \in e} \exp(-x_i/\gamma)} \right)$$

- **LSE** 误差上界为 $\gamma \ln n$, **WA**误差上界为 $\frac{\gamma \Delta x}{1 + \exp(\Delta x)/n}$
- WA误差上界差不多是LSE的一半^[1]
- 预估线长更加准确

■ 非线性——ePlace

□ $\min_{x,y} WL + \lambda N(\mathcal{S})$

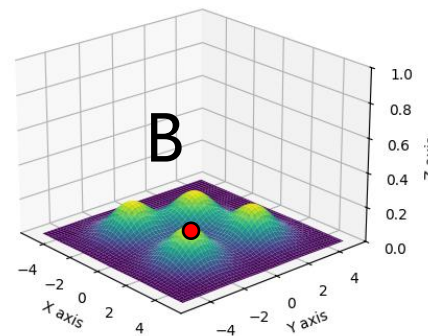
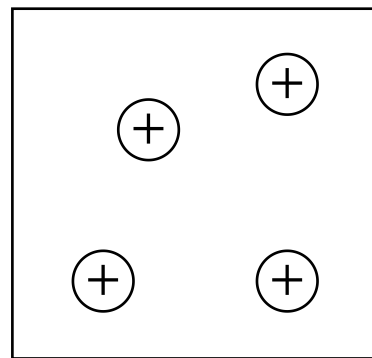
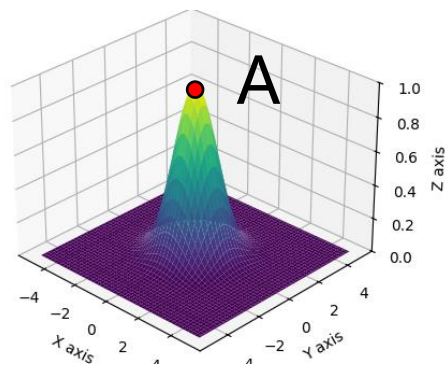
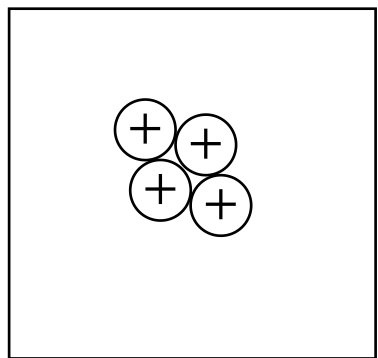
- 对于 $N(\mathcal{S}) = \sum_{i \in V} q_i \psi_i$, 建模为静电系统, 称之为 *eDensity*
- 把模块建模成正电荷, 电荷量与模块面积正相关



■ 非线性——ePlace

□ 从电荷-电场-电势 $\Rightarrow$ 模块-模块密度梯度-模块密度

- 电荷密度的高低直接影响电势的高低
- 假如多个正电荷聚集在中心
 - 若B表示一个电荷的电势 ψ_B ，A处电势则是多个电荷相互靠近后电势的累加
 - 中心区域电势高 $\Leftrightarrow$ 中心区域电荷密度高 $\Leftrightarrow$ 中心区域模块密度高
- 由此可以建立起电势能和模块密度之间的关系



■ 非线性——ePlace

□ 从电荷-电场-电势 $\Rightarrow$ 模块-模块密度梯度-模块密度

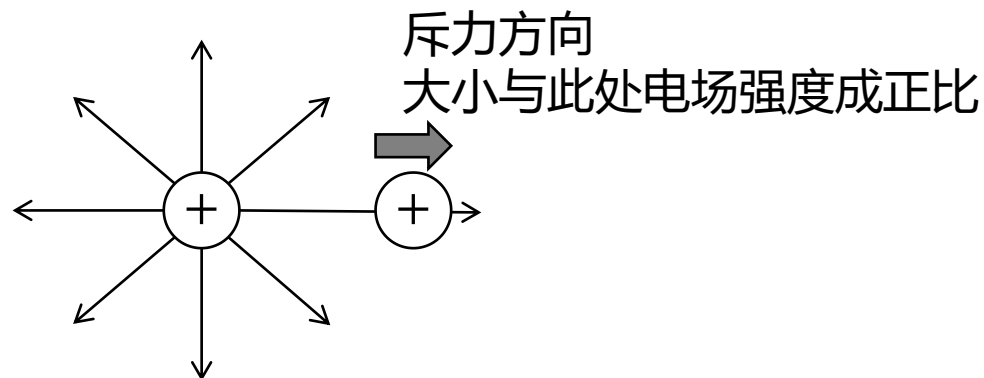
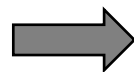
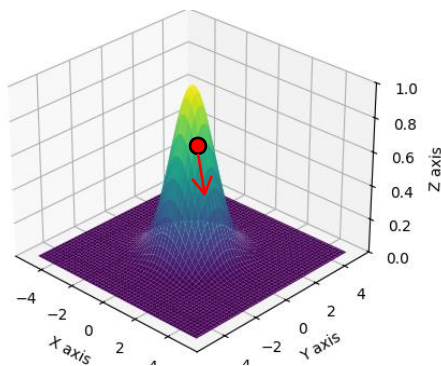
– 电场告诉电荷如何前往低电势区域

- 电场计算 $\xi(x, y) = (-\partial\psi/\partial x, -\partial\psi/\partial y)$ ，电场是电势能在 (x, y) 处的导数，描述电势能在附近的**变化强度**和**变化方向**

- 取负号是为了朝梯度下降的方向

- 模块密度梯度也是模块密度的导数

– 模块密度梯度和电场的对应关系

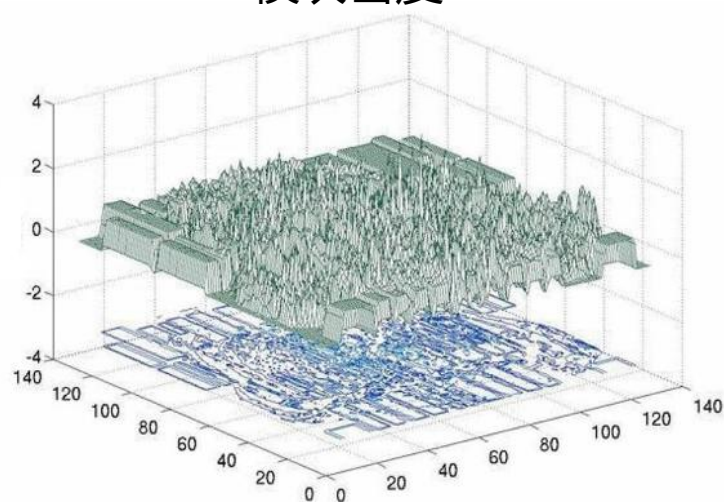


■ 非线性——ePlace

□ 利用静电系统，ePlace避免复杂的模块密度计算

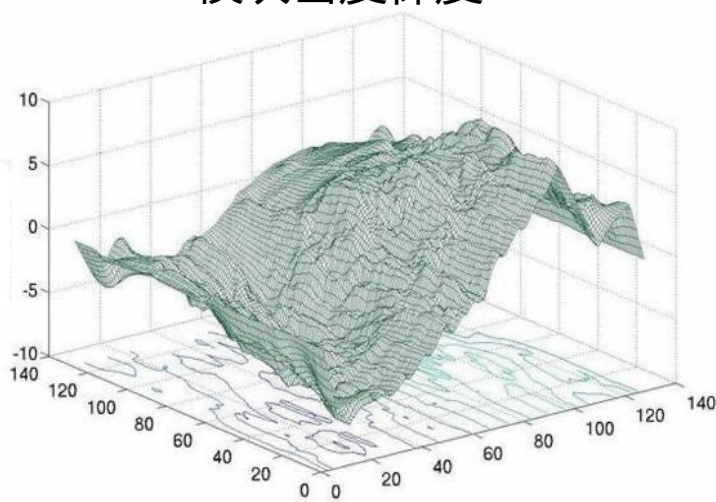
- 布局问题看做一个二维独立的静电系统，其中的电势(electric potential)和电场(electric field)的分布与系统中所有元素相关
- 模块的移动受到洛伦兹力影响 $F_i = q_i \xi_i$ ，其中 ξ_i 是模块附近局部电场强度
 - 要注意电场 ξ 带大小和方向属性

模块密度



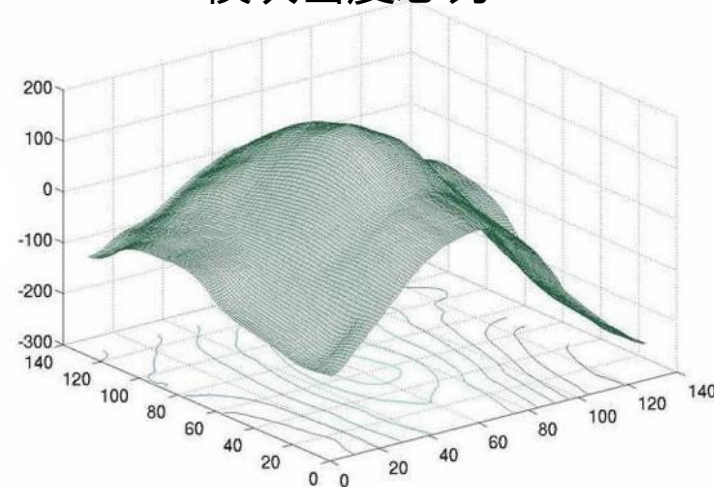
电荷密度

模块密度梯度



电场强度

模块密度惩罚

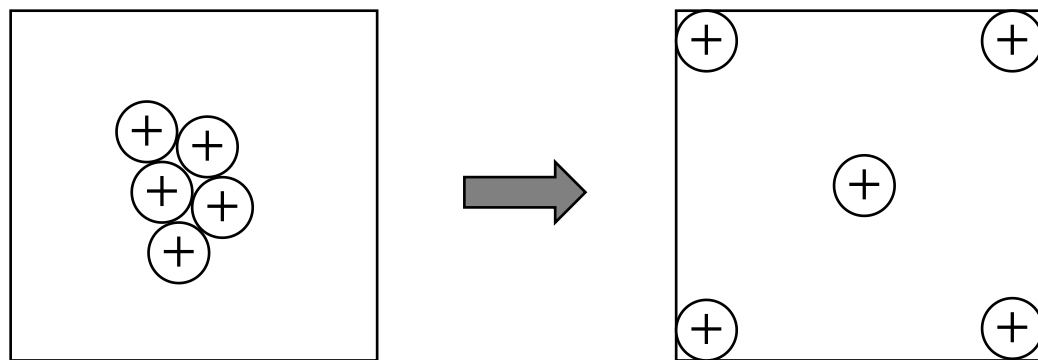


电势

■ 非线性——ePlace

□ 仅有正电荷还不够

- 想象一下，一个区域内只有正电荷
- 那么当电势能稳定不再变化时，电荷会如何分布
 - 结果是，大部分电荷会分布在区域边缘^[1]
 - 正电荷之间只有斥力，如果要维持一个电荷在中心，那么至少需要四个电荷在边缘
 - 大部分模块布置在区域的边缘，这不满足芯片布局设计一般约束

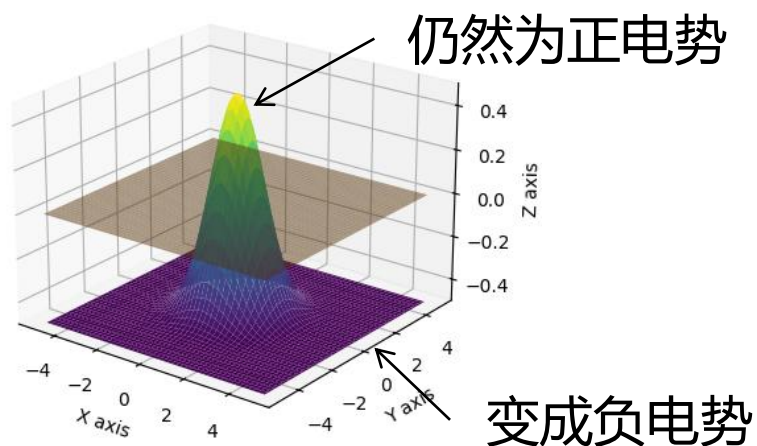


[1] 实际上是所有电荷都分布在区域的边缘，这里改动为方便直观理解。

■ 非线性——ePlace

□ 电势减去一个常量，构造负电荷区域

- 减去一个直流分量(direct-current component), 使得整个区域电势能的积分为0, 等效于在整个区域均匀的布置与正电荷等量的负电荷
- 利用正负电荷异性相吸的特性
 - 处于负电势区域的正电荷受到负电荷吸引力, 更倾向于不移动
 - 而负电荷是均匀分布的, 使得正电荷也倾向于均匀分布, 生成符合设计约束的布局



■ 非线性——ePlace

□ 最后令 $D(\mathcal{S}) = \frac{1}{2} \sum_{i \in V} q_i \psi_i$, 优化目标为 $D(\mathcal{S}) = 0$

– 把复杂的密度约束问题转化成使系统总能量为0的优化问题

➤ 因为系统能量是成对计算的, 除以2修正为单个电荷的能量

➤ 总能量为0说明电荷分布均匀, 不存在局部的聚集

– 静电系统计算公式的光滑性, 使得原优化问题非常适合使用梯度方法求解

– 其中

➤ 对于离散的电势 $\psi_v = \sum_{i \in V, i \neq v} k \frac{q_i}{r_i}$, r_i 是点电荷 v 与点电荷 i 的欧氏距离

➤ 对于连续的电势 $\psi_v = k \frac{\int g(q) dq}{r}$, dq 表示一个微元电荷

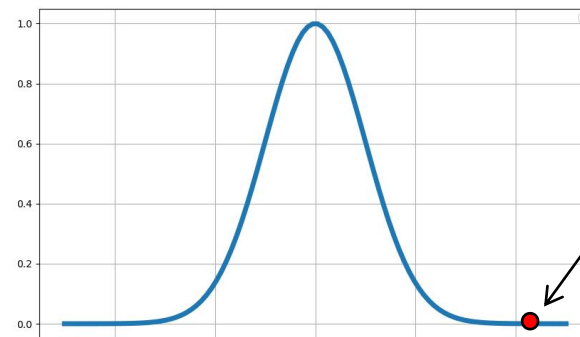
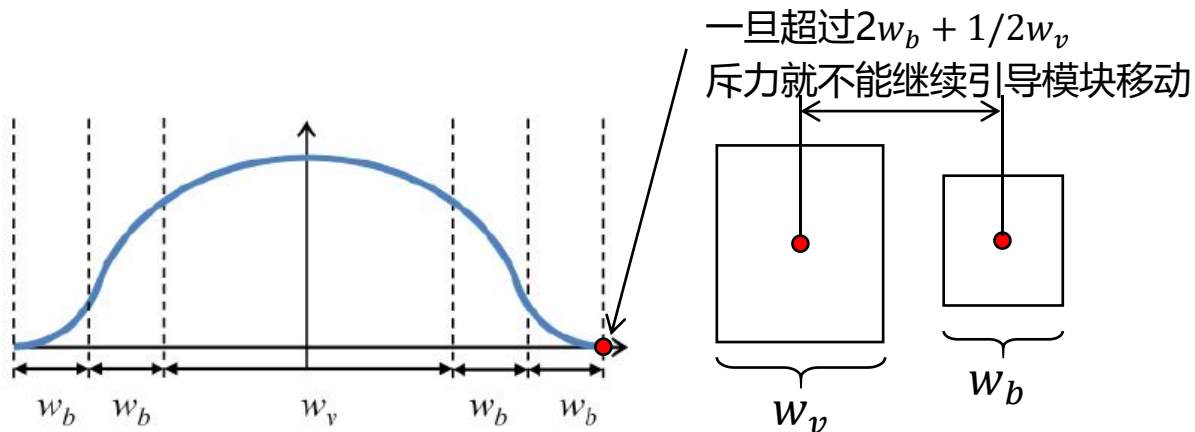
■ 非线性——ePlace

□ 对比 “钟形” 曲线光滑方法

– 在较远的距离，静电系统仍能有效引导模块向低密度区域移动

➤ 钟形曲线斥力:
$$p_x(b, v) = \begin{cases} 1 - ad_x^2 & , 0 \leq d_x \leq \frac{w_v}{2} + w_b \\ b(d_x - \frac{w_v}{2} - 2w_b)^2, & \frac{w_v}{2} + w_b \leq d_x \leq \frac{w_v}{2} + 2w_b \\ 0 & , \frac{w_v}{2} + 2w_b \leq d_x \end{cases}$$

➤ 静电系统斥力: $p_j = \sum_{i \in V, i \neq j} k \frac{q_i q_j}{r_{ij}^2}$, 其中 k 是常数, r_{ij} 表示模块 i 与模块 j 的欧氏距离



这里电场力接近0, 但永不为0

■ 非线性——ePlace

□ 全局布局算法流程

– 总览

- 采用迭代优化方法
- 采用Nesterov梯度下降法优化目标函数
 - 相比共轭梯度下降法收敛更快
- 采用动态步长 α 、动态密度惩罚因子 λ
 - 动态步长 α 加快收敛，避免陷入局部最优解
 - 动态密度惩罚因子 λ ，有利于平衡线长和密度，调整算法的优化方向。一般情况下，前期偏向线长，后期偏向密度

Algorithm 1: ePlace 的全局布局算法

Data: 初始布局 $v_0 = v_{ip}$

退出循环的最小重叠率 τ_{min}

最大迭代次数 $k_{max} = 3000$

Result: 最终布局 v_{gp}

1 初始化密度惩罚因子 λ_0 和优化步长 α_0 ;

2 for $k = 0 \rightarrow k_{max}$ do

3 $f_k \leftarrow W(v_k) + \lambda_k N(\mathcal{S})$;

4 计算线长梯度 ∇W 和元件密度 ρ_k ;

5 根据 ρ_k 计算电势和电场 (ψ_k, ξ_k) ;

6 计算总电势能的梯度 $\nabla N_k \leftarrow q\xi_k$;

7 得到目标函数的梯度 $\nabla f_k \leftarrow \nabla W_k + \lambda_k \nabla N_k$;

8 用 Nesterov 梯度下降法 $v_{k+1} \leftarrow NL_Solver(v_k, f_k, \nabla f_k, \alpha_k)$;

9 更新步长、惩罚因子、重叠密度 $\alpha_{k+1}, \lambda_{k+1}, \tau_{k+1}$;

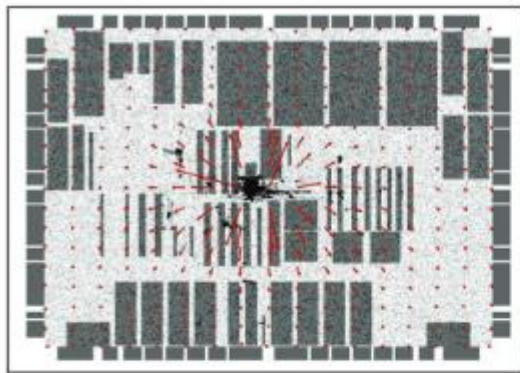
10 if $\tau_{k+1} < \tau_{min}$ then

11 $v_{gp} \leftarrow v_{k+1}$;

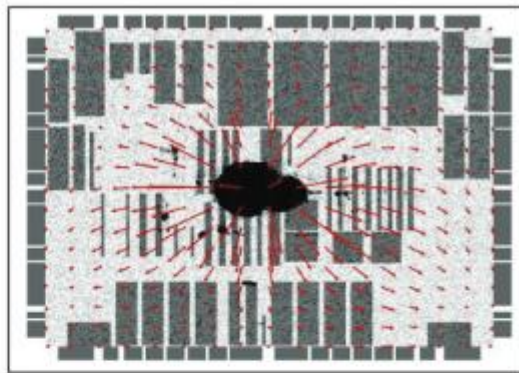
12 break;

Output: 最终布局 v_{gp}

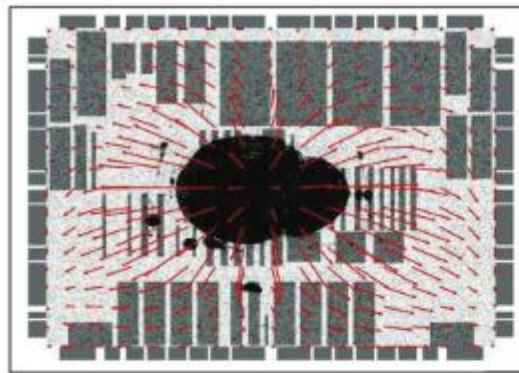
■ 非线性——ePlace



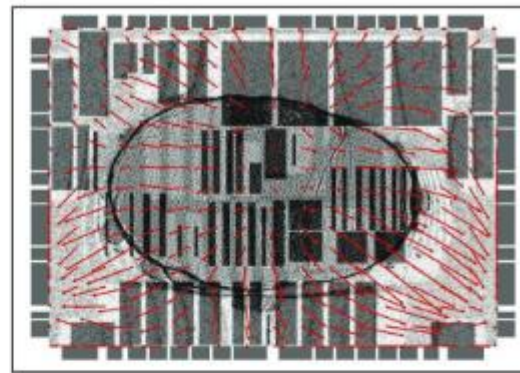
(a) iter=1, $N=1.25e12$, $\tau = 97.0\%$



(b) iter=5, $N=6.07e11$, $\tau = 93.1\%$



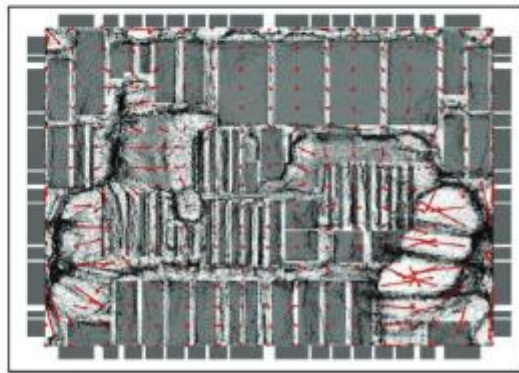
(c) iter=6, $N=2.98e11$, $\tau = 79.8\%$



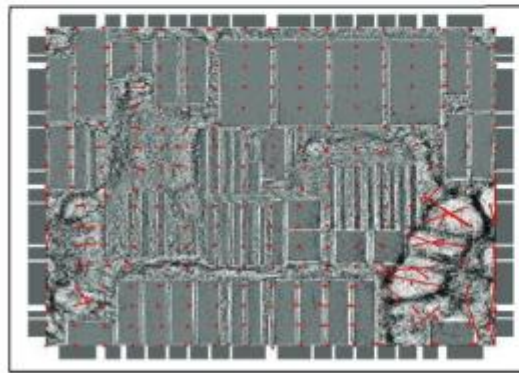
(d) iter=7, $N=5.40e10$, $\tau = 42.4\%$



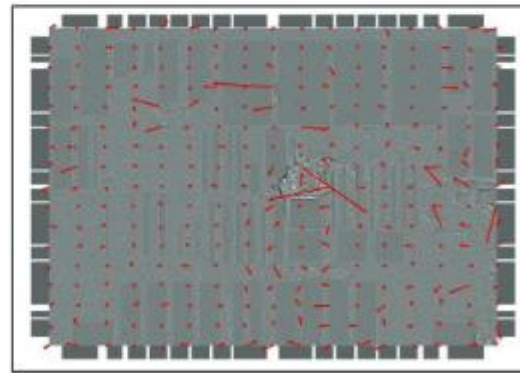
(e) iter=8, $N=2.16e10$, $\tau = 33.0\%$



(f) iter=12, $N=2.79e9$, $\tau = 27.6\%$



(g) iter=18, $N=3.07e8$, $\tau = 14.3\%$



(h) iter=60, $N=8.12e5$, $\tau = 2.54\%$

■ 非线性——ePlace

□ 总结

- 建模成静电系统相比之前模块密度计算的优势
 - 静电系统是一个完整的系统性的模型，减少数学建模的设计工作量，且鲁棒性好
 - 解决钟形曲线光滑不能给出远距离优化方向、缺少全局密度信息的问题
 - 更加简单的模块密度优化目标

■ 非线性

□ 相关开源工具介绍

– GPU加速的ePlace

- <https://github.com/limbo018/DREAMPlace>
- 提供Python接口，调用GPU加速静电系统求解

– RePlace

- <https://github.com/The-OpenROAD-Project/RePlAce>

■ 课后思考

- 假设你是中山大学东校区校园设计者，重新规划东校区建筑分布
 - 已知宿舍楼约40栋，食堂5个，教学楼、学院楼50栋，快递点1个
 - 假定各个楼为矩形，且大小已知
 - 布局区域为现在东校园的平面图
 - 尝试利用二次型或非线性布局算法框架计算建筑位置
 - **要求1**：宿舍、食堂、教学区尽可能近
 - **要求2**：建筑不能重叠
 - **要求3**：写下优化目标、约束条件以及算法流程
 - 并解释你的优化目标、约束条件的设计理由

感谢！
